Web Development Bootcamp — Revision Guide

HTML · CSS · JavaScript · Node · Express · Databases · React · Web3 / Internet Computer

Prepared for Rizwan Khalid · September 2026

Study plans, hands-on labs, original practice questions and readiness checklists. Prepared with AI assistance (Claude). Every exam and badge assessment is taken by you, under each provider's rules. Exam facts were checked against official pages in September 2026; confirm prices and dates before booking.

Contents

How to use this revision guide 5

Full topic index (find what you forgot) . . . 5

Chapter map . . . 5

Study tip . . . 6

About this guide . . . 6

Chapter 1: HTML, CSS and Bootstrap 7

What this covers . . . 7

Key concepts . . . 7

Cheatsheet . . . 9

Common mistakes . . . 10

Quick self-check . . . 10

Chapter 2: JavaScript, the DOM and jQuery 12

What this covers . . . 12

Key concepts . . . 12

Cheatsheet . . . 16

Common mistakes . . . 16

Quick self-check . . . 17

Chapter 3: Node.js, Express, EJS and APIs 18

What this covers . . . 18

Key concepts . . . 18

Cheatsheet . . . 21

Common mistakes . . . 22

Quick self-check . . . 22

Chapter 4: Databases (SQL and MongoDB) and REST 24

What this covers . . . 24

Key concepts . . . 24

Cheatsheet . . . 27

Common mistakes . . . 28

Quick self-check . . . 28

Chapter 5: Authentication and React 30

What this covers . . . 30

Key concepts . . . 30

Cheatsheet . . . 34

Common mistakes . . . 35

Quick self-check . . . 35

Chapter 6: Web3, the Internet Computer and DApps 37

What this covers . . . 37

Key concepts . . . 37

Cheatsheet . . . 41

Common mistakes . . . 42

Quick self-check . . . 43

Chapter 7: Web Design Theory 44

What this covers . . . 44

Key concepts . . . 44

Cheatsheet . . . 48

Common mistakes . . . 49

Quick self-check . . . 49

Chapter 8: Command Line, Git, GitHub and Deployment 51

What this covers . . . 51

Key concepts . . . 51

Cheatsheet . . . 55

Common mistakes . . . 56

Quick self-check . . . 57

Chapter 9: Projects and Challenges Recap 59

What this covers . . . 59

Key concepts . . . 59

Cheatsheet . . . 64

Common mistakes . . . 65

Quick self-check . . . 65

How to use this revision guide

This is a complete revision companion for *The Complete Full-Stack Web Development Bootcamp*. It restates every major topic in plain language with small, original code examples, cheatsheets and self-checks — it is not a copy of the course. Use it to jump straight to anything you have forgotten and refresh it in minutes.

Full topic index (find what you forgot)

If you need...	Go to
HTML structure, semantic tags, forms	Ch 1
CSS box model, selectors, specificity	Ch 1
Flexbox, CSS Grid, positioning	Ch 1
Responsive design, media queries	Ch 1
Bootstrap 5 grid and components	Ch 1
JavaScript types, functions, arrays, objects	Ch 2
ES6+: arrow fns, destructuring, spread, modules	Ch 2
Promises, async/await	Ch 2
DOM selection, events, manipulation	Ch 2
jQuery (legacy)	Ch 2
Node.js modules, npm, event loop	Ch 3
Express routing and middleware	Ch 3
EJS templating and partials	Ch 3
Consuming REST APIs (fetch/axios, keys)	Ch 3
SQL and PostgreSQL (pg client)	Ch 4
MongoDB and Mongoose	Ch 4
Designing/building a REST API	Ch 4
Auth: bcrypt, sessions, JWT, Passport, OAuth	Ch 5
React: JSX, props, state, hooks, forms	Ch 5
Web3, ICP, canisters, dfx	Ch 6
Motoko language	Ch 6
DBANK DeFi and token DApp	Ch 6
Deploying to the Internet Computer	Ch 6
Colour theory, typography, UI/UX	Ch 7
Command line (bash)	Ch 8
Git and GitHub	Ch 8
Deploying a site/app	Ch 8
Rebuilding the course projects	Ch 9

Chapter map

Chapter	Topics	Course area
1	HTML, CSS, Bootstrap	Front-end basics
2	JavaScript, DOM, jQuery	Browser scripting
3	Node, Express, EJS, APIs	Back-end + APIs
4	Databases (SQL, MongoDB), REST	Data layer
5	Authentication, React	Security + framework

Chapter	Topics	Course area
6	Web3, Internet Computer, DApps	Sections 37-41
7	Web Design Theory	Web Design School
8	Command Line, Git, GitHub, Deployment	Tooling + shipping
9	Projects and Challenges Recap	The build projects

Study tip

Read a chapter's `## Key concepts`, cover the `## Cheatsheet` and try to recall it, then do the `## Quick self-check`. Anything you miss, re-read that concept or rewatch just that lecture. To revise fast before an interview, read only the Cheatsheets and Common Mistakes across all chapters.

About this guide

Prepared with AI assistance (Claude) as a personal revision aid; explanations and code are original, not copied from the course. You completed the course and earned the Udemy certificate yourself.

Chapter 1: HTML, CSS and Bootstrap

What this covers

This chapter revises the front-end foundations from the early bootcamp sections: HTML5 document structure and semantic tags, building forms, the CSS box model, selectors and specificity, the two modern layout engines (flexbox and grid), positioning, responsive design with media queries, and finally Bootstrap 5 (its grid, ready-made components, and utility classes). The goal is to recall each idea fast and see a minimal example, not to re-read the lectures.

Key concepts

HTML5 document structure

Every page starts with a doctype and a root `html` element that wraps a `head` (metadata, not visible) and a `body` (the visible content). The `meta charset` and `meta viewport` tags are effectively mandatory today: the first fixes text encoding, the second makes the page scale correctly on phones.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Revision Notes</title>
  </head>
  <body>
    <h1>Hello</h1>
  </body>
</html>
```

Why it matters: without the viewport meta, mobile browsers render at a fake 980px width and shrink everything, so your media queries never fire.

Semantic tags

Semantic elements name the *role* of a region instead of using generic `div` soup. They help accessibility (screen readers), SEO, and make the markup readable. Use `header`, `nav`, `main` (only one per page), `section`, `article`, `aside`, and `footer` for structure; use `figure`/`figcaption`, `time`, and `mark` for inline meaning.

```
<header><nav>...</nav></header>
<main>
  <article>
    <h2>Post title</h2>
    <p>Body text...</p>
  </article>
  <aside>Related links</aside>
</main>
<footer>(c) 2026</footer>
```

Gotcha: a `section` should generally have a heading. If a wrapper exists only for CSS/JS and carries no meaning, a plain `div` is still correct.

Forms

A `form` collects input and submits it. Each control needs a `name` (the key sent to the server) and, for accessibility, a `label` tied via `for`/`id`. The `action` is the endpoint, `method` is usually `get` (data in the URL) or `post` (data in the body). HTML5 adds input types (`email`, `number`, `date`, `range`) and validation (`required`, `min`, `pattern`) that run before submit.

```
<form action="/subscribe" method="post">
  <label for="email">Email</label>
  <input id="email" name="email" type="email" required />
  <button type="submit">Sign up</button>
</form>
```

Why it matters: as a backend dev, remember the `name` attribute is what lands in `req.body` (with body-parser / `express.urlencoded`). No `name` means the field is silently not submitted.

The box model

Every element is a box built of four layers, from inside out: content, `padding`, `border`, and `margin`. Total rendered width depends on `box-sizing`. The default `content-box` adds padding and border *on top* of the width you set; `border-box` makes width include padding and border, which is far more predictable.

```
* {
  box-sizing: border-box;   /* width now covers padding + border */
}
.card {
  width: 300px;
  padding: 20px;            /* stays inside the 300px */
  border: 2px solid #333;
  margin: 16px;             /* space outside, not counted in width */
}
```

Gotcha: vertical margins between stacked block elements *collapse* (the larger of the two wins, they do not add up). Padding never collapses.

CSS selectors and specificity

Selectors target elements: by type (`p`), class (`.btn`), id (`#nav`), attribute (`[type="text"]`), or relationship (descendant `a b`, child `a > b`, adjacent `a + b`). When two rules set the same property, the winner is decided by specificity, scored as (inline, ids, classes, types). Higher tuple wins; ties go to whichever rule is written last.

```
p { color: gray; }              /* 0,0,0,1 */
.note { color: blue; }          /* 0,0,1,0  -> wins over p */
#lead { color: red; }           /* 0,1,0,0  -> wins over .note */
```

Why it matters: reach for a more specific selector before resorting to `!important`, which overrides everything and is painful to unwind later.

Flexbox

Flexbox lays out items along a single axis (a row or a column). Set `display: flex` on the parent; it becomes a flex container and its direct children become flex items. `justify-content` aligns along the main axis, `align-items` aligns along the cross axis, and `gap` spaces items without margins.

```
.toolbar {
  display: flex;
  justify-content: space-between;  /* main axis: push to edges */
  align-items: center;             /* cross axis: vertical center */
  gap: 12px;
}
```

Gotcha: `flex-direction: column` swaps the axes, so `justify-content` now controls vertical and `align-items` controls horizontal. Use `flex: 1` on an item to let it grow and fill remaining space.

CSS grid

Grid is two-dimensional: you define rows *and* columns and place items into cells. `grid-template-columns` sets the tracks; the `fr` unit is a fraction of free space, and `repeat()` plus `minmax()` build responsive layouts without media queries.

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 16px;
}
```

Why it matters: use flexbox for one-dimensional strips (nav bars, button rows) and grid for true 2D page layouts. The `auto-fill`/`minmax` pattern above gives a self-adjusting card grid in one line.

Positioning

`position` controls how an element is placed and whether `top`/`right`/ `bottom`/`left` apply. `static` is the default (normal flow). `relative` nudges an element from its normal spot and creates a positioning context. `absolute` removes it from flow and positions it against the nearest positioned ancestor. `fixed` pins to the viewport; `sticky` toggles between relative and fixed as you scroll.

```
.badge-wrap { position: relative; }
.badge {
  position: absolute;   /* relative to .badge-wrap */
  top: -8px;
  right: -8px;
}
.header { position: sticky; top: 0; }  /* stays on screen when scrolling */
```

Gotcha: `absolute` anchors to the closest ancestor with a non-`static` position. Forgetting to set `position: relative` on the parent makes it jump to the whole page instead.

Media queries and responsive design

Media queries apply CSS only when conditions hold, most often a viewport width. The modern habit is mobile-first: write the base styles for small screens, then use `min-width` queries to add layout as the screen grows. Combine this with fluid units (`%`, `rem`, `fr`, `clamp()`) so the design flexes between breakpoints too.

```
.container { padding: 16px; }          /* mobile base */

@media (min-width: 768px) {            /* tablet and up */
  .container { display: flex; gap: 24px; }
}
```

Why it matters: mobile-first keeps overrides additive and simpler. Pick breakpoints where the content breaks, not around specific device models.

Bootstrap 5 grid

Bootstrap is a CSS framework with a 12-column responsive grid. Wrap rows in `.container`, each row in `.row`, and size columns with `.col-*` classes using breakpoint infixes (`sm`, `md`, `lg`, `xl`, `xxl`). Bootstrap 5 dropped jQuery, so components use plain JS, and it added an `xxl` breakpoint.

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-6">Half on tablet+, full on phone</div>
    <div class="col-12 col-md-6">Second half</div>
  </div>
</div>
```

Gotcha: columns must be direct children of `.row`, and rows should sit inside a `.container` (or `.container-fluid`) or the negative row margins cause horizontal overflow.

Bootstrap 5 components and utilities

Components are pre-styled widgets: `card`, `navbar`, `modal`, `alert`, `btn`. Interactive ones (modal, dropdown, collapse) need Bootstrap's JS bundle and are wired with `data-bs-*` attributes. Utilities are tiny single-purpose classes for spacing (`m`/`p` + side + size, e.g. `mt-3`), color (`text-primary`, `bg-light`), display (`d-flex`, `d-none`), and more, so you rarely write custom CSS for common tweaks.

```
<div class="card p-3 shadow-sm">
  <h5 class="card-title mb-2">Title</h5>
  <p class="card-text text-muted">Body</p>
  <button class="btn btn-primary" data-bs-toggle="modal"
          data-bs-target="#info">Open</button>
</div>
```

Why it matters: the spacing scale runs 0-5 (roughly 0 to 3rem), and `x` means left+right, `y` means top+bottom (e.g. `px-4`, `my-2`). Learning the utility shorthand is most of what makes Bootstrap fast.

Cheatsheet

Item	Syntax / class	Purpose
Viewport meta	`width=device-width`	Enable responsive scaling
Semantic layout	`header nav main footer`	Meaningful page regions
Label a field	`<label for="id">`	Accessible form control
Predictable sizing	`box-sizing: border-box`	Width includes pad+border

Item	Syntax / class	Purpose
Class selector	`.name`	Score 0,0,1,0
Id selector	`#name`	Score 0,1,0,0
Center in flex	`justify + align-items`	Main and cross axis
Grow to fill	`flex: 1`	Take remaining space
Responsive grid	`repeat(auto-fill,minmax())`	Self-adjusting columns
Anchor child	parent `position: relative`	Base for `absolute`
Stick on scroll	`position: sticky; top: 0`	Pin while scrolling
Breakpoint up	`@media (min-width: 768px)`	Mobile-first query
BS grid column	`col-12 col-md-6`	12-col responsive
BS spacing	`mt-3`, `px-4`, `my-2`	Margin/padding scale 0-5
BS JS trigger	`data-bs-toggle`	Modal, dropdown, collapse

Common mistakes

- Omitting the viewport meta, so media queries never trigger on mobile.
- Leaving `box-sizing` at default, then fighting widths that overflow once padding is added.
- Forgetting the `name` attribute, so the field is not submitted to the server (empty `req.body`).
- Reaching for `!important` instead of a slightly more specific selector.
- Mixing up flex axes: assuming `align-items` is always vertical, even after `flex-direction: column`.
- Positioning `absolute` without a positioned ancestor, so it anchors to the whole page.
- Nesting Bootstrap columns that are not direct children of `.row`, or a `.row` without a container, causing horizontal overflow.
- Assuming Bootstrap 5 needs jQuery (it does not) or loading the old `data-toggle` attributes (v5 uses `data-bs-toggle`).

Quick self-check

1. What are the two `meta` tags almost every page needs, and what does each do?
2. In `content-box` vs `border-box`, which includes padding inside the declared width?
3. Rank these by specificity: `p`, `.note`, `#lead`.
4. Which layout tool is one-dimensional and which is two-dimensional?
5. For `position: absolute` to anchor to a parent, what must the parent have?
6. What does the `fr` unit mean in CSS grid?
7. Why is mobile-first (`min-width` queries) usually preferred?

8. In Bootstrap, what does `col-12 col-md-6` produce across screen sizes?
9. What does the Bootstrap class `my-2` set?
10. Which attribute prefix wires Bootstrap 5 interactive components?

Answers

1. `meta charset` sets text encoding; `meta viewport` with `width=device-width` enables correct mobile scaling.
2. `border-box` includes padding (and border) inside the declared width; `content-box` adds them on top.
3. `#lead` (0,1,0,0) > `.note` (0,0,1,0) > `p` (0,0,0,1).
4. Flexbox is one-dimensional (a row or column); grid is two-dimensional (rows and columns together).
5. A non-`static` position, typically `position: relative`, on the nearest ancestor.
6. A fraction of the remaining free space in the grid container.
7. Overrides stay additive: base styles cover small screens, and you add layout as width grows, which is simpler to reason about.
8. Full width (one column) on phones, and two half-width columns from the `md` breakpoint (768px) upward.
9. Vertical margin (top and bottom) at scale step 2.
10. The `data-bs-*` prefix, for example `data-bs-toggle` and `data-bs-target`.

Chapter 2: JavaScript, the DOM and jQuery

What this covers

This chapter revises the JavaScript sections of the bootcamp: the language fundamentals (types, variable declarations, functions, arrays, objects, loops), the modern ES6+ features you use every day (arrow functions, template literals, destructuring, spread, modules, promises, async/await), how JavaScript talks to the page through the DOM (selecting elements, listening for events, changing the document), and a short tour of jQuery (selection, events, effects). jQuery is treated as legacy: worth recognising in old code, but not what you reach for in new projects. Version note: everything here targets modern browsers and Node 18+ as of 2026, where ES modules and async/await are fully supported.

Key concepts

Types and coercion

JavaScript has seven primitive types (`string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`) plus `object` (which includes arrays and functions). Primitives are compared by value; objects by reference. The trap is coercion: `==` converts operands before comparing, so prefer `===`.

```
typeof "hi";        // "string"
typeof null;        // "object"  (a historical bug, but stable)
1 == "1";           // true  -- coerced
1 === "1";          // false -- strict, no coercion
[] == false;        // true  -- avoid; use === and Boolean() explicitly
```

Why it matters: almost every "weird JavaScript" bug is loose equality or an unexpected coercion. Default to `===` and convert types on purpose with `Number()`, `String()`, `Boolean()`.

var, let and const

`var` is function-scoped and hoisted (initialised as `undefined` before its line runs). `let` and `const` are block-scoped and live in a "temporal dead zone" until declared, so using them early throws. `const` forbids reassignment of the binding, not mutation of the object it points at.

```
function scopeDemo() {
  if (true) {
    var a = 1;    // leaks to the whole function
    let b = 2;    // only inside this block
  }
  console.log(a); // 1
  // console.log(b); // ReferenceError
}

const nums = [1, 2];
nums.push(3);     // fine -- mutating the array
// nums = [4];    // TypeError -- reassigning the binding
```

Gotcha: reach for `const` by default, `let` when you must reassign, and avoid `var` in new code.

Functions

Three ways to define them: declarations (hoisted, callable before their line), expressions (assigned to a variable), and arrow functions (compact, no own `this`). Functions are first-class values, so you can pass them around and return them.

```
function add(a, b) { return a + b; }      // declaration
const mul = function (a, b) { return a * b; }; // expression
const sub = (a, b) => a - b;              // arrow

function greet(name = "world") {          // default parameter
  return `Hello, ${name}`;
}
```

Why it matters: passing functions as arguments (callbacks) is the backbone of array methods, event handlers, and async code.

Arrays

Ordered, zero-indexed lists. The high-value methods are the iterators that return new data instead of mutating: `map`, `filter`, `reduce`, `find`, `some`, `every`. Mutating methods (`push`, `pop`, `splice`, `sort`) change in place.

```
const nums = [5, 2, 8, 1];
const doubled = nums.map(n => n * 2);        // [10, 4, 16, 2]
const evens = nums.filter(n => n % 2 === 0); // [2, 8]
const total = nums.reduce((sum, n) => sum + n, 0); // 16
const first = nums.find(n => n > 4);         // 5
```

Gotcha: `sort()` compares as strings by default, so `[10, 2].sort()` gives `[10, 2]` unwanted. Pass a comparator: `.sort((a, b) => a - b)`.

Objects

Key-value collections. Access with dot or bracket notation; bracket lets you use dynamic or non-identifier keys. Methods are just function-valued properties, and `this` inside a normal method refers to the object it was called on.

```
const user = {
  name: "Rizwan",
  role: "developer",
  greet() { return `Hi, I'm ${this.name}`; },
};

user.name;          // "Rizwan"
user["role"];       // "developer"
const key = "name";
user[key];          // "Rizwan"
Object.keys(user);  // ["name", "role", "greet"]
```

Why it matters: objects model almost everything (config, API responses, records). Knowing `Object.keys/values/entries` makes iterating them easy.

Loops

`for` for counting, `for...of` for values of an iterable (array, string, Map), `for...in` for keys of an object (use sparingly), and `while` for condition-driven repetition. For arrays, the iterator methods above are usually clearer than a manual loop.

```
for (let i = 0; i < 3; i++) { /* 0,1,2 */ }

for (const n of [10, 20, 30]) { /* 10,20,30 */ }

for (const key in { a: 1, b: 2 }) { /* "a","b" */ }
```

Gotcha: `for...in` also walks inherited enumerable keys and gives string indexes for arrays. Use `for...of` or `Object.keys()` instead.

Arrow functions and this

Arrows have no own `this`, `arguments`, or `prototype`; they inherit `this` from the enclosing scope. That makes them ideal for callbacks inside a method, and wrong as object methods that need their own `this`.

```
const timer = {
  count: 0,
  start() {
    setInterval(() => { this.count++; }, 1000); // this = timer, correct
  },
};
```

Gotcha: a `const obj = { bad: () => this.x }` arrow method does not see the object as `this` -- use a normal method there.

Template literals

Backtick strings with `${}` interpolation and real newlines. They replace clumsy string concatenation and make small HTML fragments readable.

```
const name = "Rizwan";
const items = ["a", "b"];
```

```
const html = `
  <h1>Hi ${name}</h1>
  <p>${items.length} items</p>
`;
```

Why it matters: cleaner than `"Hi " + name + "..."` and safer to read, so fewer misplaced-quote bugs.

Destructuring

Pull values out of arrays or objects into variables in one step. Works in parameters too, which is the common way to receive an options object.

```
const [first, second] = [10, 20];        // first=10, second=20
const { name, role = "guest" } = user;   // with a default

function connect({ host, port = 5432 }) { // destructured param
  return `${host}:${port}`;
}
```

Gotcha: destructuring `undefined` throws. Guard with a default: `function f({ a } = {}) {}`.

Spread and rest

The `...` operator spreads an iterable into elements/args, or collects the rest into an array/object. It is the idiomatic way to copy and merge without mutating.

```
const a = [1, 2];
const b = [...a, 3];              // [1, 2, 3] -- shallow copy + add
const merged = { ...user, active: true }; // clone object + field

function sum(...nums) {           // rest gathers args
  return nums.reduce((s, n) => s + n, 0);
}
```

Gotcha: spread copies are shallow -- nested objects/arrays are still shared references.

Modules

ES modules split code into files with `export` and `import`. Each file has its own scope; nothing is global. Named exports for many values, one `default` export per file. In HTML use `<script type="module">`; in Node use `.mjs` or `"type": "module"` in package.json.

```
// math.js
export const PI = 3.14159;
export function area(r) { return PI * r * r; }
export default function circumference(r) { return 2 * PI * r; }

// app.js
import circumference, { PI, area } from "./math.js";
```

Why it matters: modules replace the old pattern of many global scripts and are what every bundler and framework expects.

Promises

A promise represents a value that arrives later; it is `pending`, then `fulfilled` or `rejected`. Chain with `.then()` and handle errors with `.catch()`. `Promise.all` runs several in parallel and resolves when all finish.

```
fetch("/api/users")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error("failed:", err));

Promise.all([fetch("/a"), fetch("/b")]).then(([ra, rb]) => { /* both */ });
```

Gotcha: a rejected promise with no `.catch()` becomes an unhandled rejection. Always terminate a chain with error handling.

async / await

Syntactic sugar over promises: `await` pauses inside an `async` function until a promise settles, so async code reads top-to-bottom. Wrap in `try/catch` for errors.

```
async function loadUsers() {
  try {
    const res = await fetch("/api/users");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error("failed:", err);
    return [];
  }
}
```

Gotcha: awaiting in a loop serialises requests. For independent calls, start them together and `await Promise.all([...])`.

The DOM: selecting elements

The DOM is the browser's live object tree of the page. Modern selection uses `querySelector` (first match) and `querySelectorAll` (a static NodeList) with CSS selectors.

```
const title = document.querySelector("#title");     // by id
const items = document.querySelectorAll(".item");   // NodeList
const nav = document.querySelector("nav > ul");     // any CSS selector
items.forEach(el => console.log(el.textContent));
```

Gotcha: `querySelectorAll` returns a static NodeList, not a live one and not a real array -- spread it (`[...items]`) if you need `map`/`filter`.

The DOM: events

Attach handlers with `addEventListener`. The handler gets an `event` object; `event.target` is the element that triggered it. Use event delegation (one listener on a parent) for lists that change.

```
const btn = document.querySelector("#save");
btn.addEventListener("click", (event) => {
  event.preventDefault();          // stop default (e.g. form submit)
  console.log("clicked", event.target);
});

// delegation: one listener handles many children
list.addEventListener("click", (e) => {
  if (e.target.matches("li")) console.log("item:", e.target.textContent);
});
```

Why it matters: `addEventListener` allows many handlers and clean removal, unlike the old `onclick` attribute.

The DOM: manipulation

Change content with `textContent` (safe, plain text) or `innerHTML` (parses HTML -- risky with untrusted input). Change styling with `classList`. Create and insert nodes with `createElement` and `append`.

```
el.textContent = "Safe text";
el.classList.add("active");
el.classList.toggle("open");
el.setAttribute("aria-hidden", "true");

const li = document.createElement("li");
li.textContent = "New item";
list.append(li);
```

Gotcha: setting `innerHTML` from user input is an XSS hole. Prefer `textContent`, or sanitise before inserting HTML.

jQuery basics (legacy)

jQuery wraps DOM work in a terse `$()` API that smoothed over old browser differences. Those differences are gone, so new code uses the native DOM. You still meet jQuery in legacy projects and WordPress themes. Selection mirrors CSS; `.on()` binds events; effects like `.fadeIn()` animate.

```
$("#save").on("click", function () {   // select + event
  $(this).addClass("active");          // manipulation
```

```
  $(".panel").fadeIn(200);            // effect
  $("#list").append("<li>New</li>");
});
```

Why it matters: recognise it, maintain it, but reach for `querySelector` and `addEventListener` in anything new -- native APIs now cover what jQuery was invented to fix.

Cheatsheet

Task	Modern JS	jQuery (legacy)
Select one	`document.querySelector(sel)`	`$(sel)`
Select many	`document.querySelectorAll(sel)`	`$(sel)`
On click	`el.addEventListener("click", fn)`	`$(el).on("click", fn)`
Set text	`el.textContent = s`	`$(el).text(s)`
Set HTML	`el.innerHTML = s`	`$(el).html(s)`
Add class	`el.classList.add("x")`	`$(el).addClass("x")`
Toggle class	`el.classList.toggle("x")`	`$(el).toggleClass("x")`
Append node	`parent.append(child)`	`$(parent).append(child)`
Show/hide	`el.style.display = "none"`	`$(el).hide()` / `.show()`
Fetch data	`await fetch(url)`	`$.ajax({ url })`

Feature	Syntax
Arrow fn	`const f = (a, b) => a + b;`
Template	`` ` Hi ${name} ` ``
Destructure	`const { a, b } = obj;`
Spread copy	`const c = [...arr];`
Rest param	`function f(...args) {}`
Default param	`function f(x = 1) {}`
Named export	`export function f() {}`
Default import	`import f from "./m.js";`
Promise chain	`p.then(fn).catch(fn)`
Await	`const x = await p;`
Map array	`arr.map(x => x * 2)`
Filter array	`arr.filter(x => x > 0)`
Reduce array	`arr.reduce((s, x) => s + x, 0)`

Common mistakes

- Using `==` instead of `===`, then chasing a coercion bug.
- Declaring with `var` and leaking a variable out of its block.
- Reassigning a `const` object versus mutating it -- only reassign throws.
- Using an arrow function as an object method that needs its own `this`.
- Forgetting `sort()` compares as strings; always pass a comparator.
- Treating a NodeList like an array; spread it first for `map`/`filter`.
- Setting `innerHTML` from user input and opening an XSS hole.
- Leaving a promise chain without `.catch()` (unhandled rejection).
- Awaiting inside a loop for independent calls instead of `Promise.all`.

- Reaching for jQuery in new code when native DOM APIs already suffice.

Quick self-check

1. What does `typeof null` return, and why is that surprising?
2. Give the one-line rule for choosing `const`, `let`, or `var`.
3. Why does `[10, 2].sort()` produce `[10, 2]`, and how do you fix it?
4. When is an arrow function the wrong choice for a method?
5. What is the difference between `textContent` and `innerHTML`?
6. Why prefer event delegation for a list whose items change?
7. What does `Promise.all` do, and when do you use it over sequential awaits?
8. Name two things ES modules give you over old global `<script>` tags.
9. Is `querySelectorAll`'s result a real array? What do you do about it?
10. Why is jQuery considered legacy in 2026?

Answers

1. `"object"` -- a long-standing bug in the language kept for compatibility.

Check for null explicitly with `x === null`.

2. Default to `const`; use `let` only when you must reassign; avoid `var`.
3. `sort()` converts elements to strings, so "10" sorts before "2". Pass a

numeric comparator: `.sort((a, b) => a - b)`.

4. When the method needs its own `this` bound to the object; arrows inherit

`this` from the surrounding scope instead.

5. `textContent` sets plain text safely; `innerHTML` parses the string as

HTML, which is powerful but an XSS risk with untrusted input.

6. One listener on the parent handles current and future children and costs

less than binding a handler to every item.

7. It runs several promises in parallel and resolves when all fulfil (or

rejects on the first failure). Use it for independent async calls.

8. Per-file scope (no globals) and explicit `import`/`export` dependencies;

also static analysis and tree-shaking by bundlers.

9. No -- it is a static NodeList. Spread it into an array (`[...nodes]`) or

use `Array.from` to get `map`/`filter`/`reduce`.

10. Native APIs (`querySelector`, `addEventListener`, `fetch`, `classList`)

and modern syntax now cover what jQuery was built to smooth over, so it adds weight without unique value in new projects.

Chapter 3: Node.js, Express, EJS and APIs

What this covers

This chapter maps to the back-end sections of the bootcamp: running JavaScript outside the browser with Node.js, building an HTTP server with Express (routing, middleware, static files), rendering dynamic HTML with EJS templates, reading submitted form data, and calling third-party REST APIs from the server. Version assumptions (as of 2026): Node.js 20+ LTS, Express 4.x with a note on 5.x, and EJS 3.x. Native `fetch` is stable in Node 18+, so it is used alongside axios.

Key concepts

Node.js and the module system

Node runs JavaScript on the V8 engine outside the browser, so there is no `window` or `document` but there are `process`, `fs`, and network APIs. Node has two module styles: CommonJS (`require`/`module.exports`) and ES Modules (`import`/`export`). CommonJS is still the bootcamp default; ESM is enabled by `"type": "module"` in `package.json` or a `.mjs` extension.

```
// math.js (CommonJS)
function add(a, b) {
  return a + b;
}
module.exports = { add };

// app.js
const { add } = require("./math");
console.log(add(2, 3)); // 5
```

Why it matters: mixing the two styles in one project is the most common "Cannot use import statement outside a module" error. Pick one per project.

npm and package.json

`npm` installs packages into `node_modules` and records them in `package.json`. Run `npm init -y` to create the manifest, then `npm install express` to add a dependency. `dependencies` ship to production; `devDependencies` (installed with `npm install --save-dev nodemon`) are only for development. `package-lock.json` pins exact versions so installs are reproducible.

```
{
  "name": "my-app",
  "type": "commonjs",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

Gotcha: the ^ caret allows minor and patch upgrades. Commit `package-lock.json` so teammates and CI get identical trees.

The event loop and non-blocking I/O

Node is single-threaded for your JavaScript but offloads I/O (disk, network, timers) to the libuv thread pool and the OS. When I/O finishes, its callback is queued and run when the call stack is clear. This is why Node handles many concurrent connections cheaply, and why a long synchronous loop blocks everything.

```
console.log("first");
setTimeout(() => console.log("third (async)"), 0);
Promise.resolve().then(() => console.log("second (microtask)"));
console.log("first-and-a-half");
// Order: first, first-and-a-half, second, third
```

Why it matters: promises (microtasks) run before timers (macrotasks). Never put heavy CPU work on the main thread of a server; use async APIs or a worker.

Express: a minimal server

Express is a thin routing and middleware layer over Node's `http` module. You create an app, attach route handlers, and listen on a port. Each handler receives `req` (the request) and `res` (the response).

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Hello from Express");
});

app.listen(3000, () => console.log("Listening on http://localhost:3000"));
```

Gotcha: `res.send()`, `res.json()`, or `res.render()` must be called once per request. Calling two of them throws "Cannot set headers after they are sent".

Routing: methods, paths, and params

Routes match an HTTP method plus a path. Path segments starting with `:` become route parameters on `req.params`; the query string (`?q=cats`) lands on `req.query`. Order matters: Express tries routes top to bottom and uses the first match.

```
app.get("/users/:id", (req, res) => {
  res.send(`User ${req.params.id}, sort=${req.query.sort}`);
});

app.post("/users", (req, res) => {
  res.status(201).send("Created");
});
```

Why it matters: put specific routes (`/users/new`) before dynamic ones (`/users/:id`), or `:id` will greedily capture "new".

Middleware and the request pipeline

Middleware are functions with the signature `(req, res, next)` that run in order for matching requests. They can inspect or mutate `req`/`res`, end the response, or call `next()` to pass control on. This is how logging, parsing, auth, and error handling are layered.

```
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // forget this and the request hangs
});

// Error-handling middleware has FOUR args
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send("Something broke");
});
```

Gotcha: an error handler is identified by its four parameters. Drop the unused `next` and Express treats it as a normal middleware.

Serving static files

`express.static` serves a folder of assets (CSS, images, client JS) directly. Files are exposed at the URL path relative to that folder root, not the folder name.

```
app.use(express.static("public"));
// public/styles/main.css -> http://localhost:3000/styles/main.css
```

Why it matters: never hardcode `public/` into links; the middleware strips it. Use a mount path (`app.use("/static", express.static("public"))`) if you want a prefix.

EJS: rendering dynamic HTML

EJS (Embedded JavaScript) lets you inject server data into HTML templates. Set the view engine, keep templates in a `views/` folder with a `.ejs` extension, and call `res.render("name", dataObject)`. `<%= %>` outputs an escaped value, `<%- %>` outputs raw (unescaped) HTML, and `<% %>` runs logic without output.

```
app.set("view engine", "ejs"); // views/ is the default folder

app.get("/", (req, res) => {
  res.render("index", { title: "Home", items: ["a", "b"] });
});
```

```
<!-- views/index.ejs -->
<h1><%= title %></h1>
<ul>
  <% items.forEach(item => { %>
    <li><%= item %></li>
  <% }) %>
</ul>
```

Gotcha: referencing a variable you did not pass throws `X is not defined`. Use `typeof x !== "undefined"` or pass a default.

EJS partials and layout reuse

Repeated markup (header, footer, nav) goes into partial files included with `<%- include("partials/header") %>`. Partials can receive data by passing a second argument, which keeps them reusable.

```
<!-- views/partials/header.ejs -->
<header><nav><a href="/">Home</a></nav></header>

<!-- views/index.ejs -->
<%- include("partials/header") %>
<main><%= content %></main>
<%- include("partials/footer") %>
```

Why it matters: use `<%-` (raw) for includes. `<%=` would escape the partial's HTML and print angle brackets as text.

Parsing submitted forms

An HTML form POSTs URL-encoded data by default. Express does not read the request body until you add a body-parsing middleware. `express.urlencoded` handles form fields; `express.json` handles JSON payloads (typical for API clients). Parsed fields appear on `req.body`.

```
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

app.post("/submit", (req, res) => {
  const { username, email } = req.body;
  res.send(`Got ${username} / ${email}`);
});
```

```
<form action="/submit" method="POST">
  <input name="username" />
  <input name="email" type="email" />
  <button>Send</button>
</form>
```

Gotcha: the input's `name` attribute becomes the `req.body` key. Without `name`, the field is never submitted. `extended: true` allows nested objects and arrays via the `qs` library.

Consuming REST APIs from the server

Calling APIs from the back end (not the browser) keeps API keys secret and avoids CORS issues. Use native `fetch` (built into Node 18+) or the axios library. `fetch` does not reject on HTTP error statuses, so check `res.ok` yourself; axios rejects on 4xx/5xx automatically.

```
// Native fetch
app.get("/weather", async (req, res) => {
  const r = await fetch("https://api.example.com/now?city=London");
  if (!r.ok) return res.status(r.status).send("API error");
  const data = await r.json();
  res.render("weather", { temp: data.temp });
});
```

```
// axios (npm install axios)
const axios = require("axios");
const { data } = await axios.get("https://api.example.com/now", {
  params: { city: "London" }
});
```

Why it matters: always `await` and wrap in `try/catch`. An unhandled rejection in a route can crash the process.

API keys, GET vs POST, and status codes

Most APIs need a key sent as a query parameter or, more securely, an `Authorization` header. Keep keys in environment variables (`process.env.KEY`) loaded from a `.env` file with the `dotenv` package, and never commit `.env`. Use GET to read and POST to create; send POST bodies as JSON with the right `Content-Type`.

```
require("dotenv").config(); // loads .env into process.env

const r = await fetch("https://api.example.com/items", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${process.env.API_KEY}`
  },
  body: JSON.stringify({ name: "Widget" })
});
```

Gotcha: a 401 means the key is missing or wrong; 403 means the key is valid but not allowed; 429 means you are rate-limited. Read the status before the body.

Cheatsheet

Task	Command / syntax
Init project	`npm init -y`
Add dependency	`npm install express`
Add dev dependency	`npm install --save-dev nodemon`
Run with auto-reload	`npx nodemon app.js`
Import (CommonJS)	`const x = require("./x")`
Export (CommonJS)	`module.exports = { x }`
Create app	`const app = express()`
GET route	`app.get("/path", (req, res) => {})`
Route param	`req.params.id`
Query string	`req.query.sort`
Form / JSON body	`req.body.field`
Global middleware	`app.use(fn)`
Pass control	`next()`
Static files	`app.use(express.static("public"))`
Parse form	`app.use(express.urlencoded({extended:true}))`
Parse JSON	`app.use(express.json())`
Set EJS	`app.set("view engine", "ejs")`
Render view	`res.render("index", { data })`
EJS escaped output	`<%= value %>`
EJS raw output	`<%- html %>`
EJS logic	`<% if (x) { %> ... <% } %>`
EJS include	`<%- include("partials/header") %>`
Set status	`res.status(404).send("Not found")`
Send JSON	`res.json({ ok: true })`

Task	Command / syntax
Redirect	`res.redirect("/login")`
Env variable	`process.env.API_KEY`
Load .env	`require("dotenv").config()`

Common mistakes

- Forgetting `next()` in middleware, so the request hangs until it times out.
- Calling `res.send` and `res.render` in the same handler (headers already sent).
- Missing `express.urlencoded`/`express.json`, so `req.body` is `undefined`.
- Placing `/users/:id` before `/users/new`, letting `:id` swallow "new".
- Using `<%=` for an EJS `include`, which escapes and prints the partial's HTML.
- Not checking `res.ok` with `fetch`; a 404 or 500 is treated as success.
- Blocking the event loop with a synchronous loop or `fs.readFileSync` in a route.
- Committing `.env` or hardcoding API keys into source.
- Mixing `require` and `import` in one file without setting the module type.
- Forgetting the `name` attribute on form inputs, so fields never reach the server.

Quick self-check

1. Which HTTP status code should a successful resource-creation POST return?
2. What is the difference between `<%= %>` and `<%- %>` in EJS?
3. Why does `req.body` come back `undefined` for a submitted form?
4. In what order do a `setTimeout(fn, 0)` callback and a resolved `Promise.then` callback run?
5. What does `express.static("public")` change about the URL of `public/img/logo.png`?
6. How does native `fetch` signal an HTTP 500, and how does that differ from axios?
7. What distinguishes an Express error-handling middleware from a normal one?
8. Where should an API key live, and why not in the source file?
9. What does calling `next()` do inside middleware?

Answers

1. `201 Created` (optionally with a `Location` header for the new resource).
2. `<%= %>` HTML-escapes the value; `<%- %>` outputs it raw, used for includes and trusted HTML.
3. No body-parsing middleware is registered; add `express.urlencoded({ extended: true })`.
4. The promise callback (a microtask) runs first, then the timer callback (a macrotask).

5. It is served at `/img/logo.png`; the `public` folder name is stripped from the URL.
6. `fetch` still resolves on a 500, so you must check `res.ok`; axios rejects the promise on 4xx/5xx.
7. It declares four parameters `(err, req, res, next)`; the extra `err` marks it as an error handler.
8. In an environment variable loaded from an uncommitted `.env` file, so secrets are not exposed in version control or to the browser.
9. It passes control to the next matching middleware or route handler in the pipeline.

Chapter 4: Databases (SQL and MongoDB) and REST

What this covers

This chapter maps to the course sections on SQL and relational databases, PostgreSQL with the `node-postgres` (`pg`) client, NoSQL with MongoDB, the Mongoose ODM, and REST API design. The goal is to be able to model data in both a relational and a document store, run CRUD from Node, and expose that data through a clean REST API in Express. Version assumptions (as of 2026): PostgreSQL 16/17, `pg` 8.x, MongoDB 7/8, Mongoose 8.x, Express 5 (the default on `npm install express`), Node 20+.

Key concepts

Relational model: tables, rows, keys

A relational database stores data as tables (relations). Each row is a record, each column has a fixed type. A **primary key** uniquely identifies a row; a **foreign key** in one table points at the primary key of another, which is how relationships are enforced. The schema is fixed up front, and the database rejects data that breaks it.

```
CREATE TABLE authors (
  id    SERIAL PRIMARY KEY,
  name  VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE
);

CREATE TABLE books (
  id         SERIAL PRIMARY KEY,
  title      VARCHAR(200) NOT NULL,
  author_id  INTEGER REFERENCES authors(id),
  published  DATE
);
```

Why it matters: constraints (`NOT NULL`, `UNIQUE`, `REFERENCES`) push data-integrity rules into the database, so bad data cannot be written even if the app has a bug.

SQL CRUD

Four statements cover create, read, update, delete. Always filter `UPDATE` and `DELETE` with a `WHERE` clause or you change every row.

```
INSERT INTO authors (name, email)
VALUES ('Ada Byron', 'ada@example.com');

SELECT id, name FROM authors WHERE name LIKE 'A%';

UPDATE authors SET email = 'ada@calc.org' WHERE id = 1;

DELETE FROM authors WHERE id = 1;
```

Gotcha: `SELECT *` returns whatever the schema currently has and pulls extra bytes over the wire. Name the columns you actually need.

Filtering, sorting, aggregating

Beyond plain selects, SQL shapes result sets with `WHERE`, `ORDER BY`, `LIMIT`, and aggregate functions grouped by `GROUP BY`. `HAVING` filters on an aggregate (you cannot use `WHERE` for that).

```
SELECT author_id, COUNT(*) AS book_count
FROM books
GROUP BY author_id
HAVING COUNT(*) > 2
ORDER BY book_count DESC
LIMIT 10;
```

Why it matters: pushing aggregation into SQL is far faster than pulling every row into Node and counting there.

Joins

A join combines rows from two tables on a matching condition, usually a foreign key. An **INNER JOIN** keeps only rows that match on both sides. A **LEFT JOIN** keeps every row from the left table and fills NULL where the right side has no match (useful to find records with no related rows).

```
SELECT b.title, a.name AS author
FROM books b
INNER JOIN authors a ON a.id = b.author_id;

-- authors with zero books:
SELECT a.name
FROM authors a
LEFT JOIN books b ON b.author_id = a.id
WHERE b.id IS NULL;
```

Gotcha: forgetting the `ON` condition produces a cross join (every row paired with every other), which explodes the result size.

PostgreSQL with node-postgres (pg)

`pg` is the standard PostgreSQL driver for Node. In production use a **connection Pool**, not a single client: the pool reuses a set of TCP connections instead of opening one per request. Pass values as a parameter array (`$1`, `$2`) so `pg` escapes them — never build SQL by string concatenation.

```
import pg from "pg";

const pool = new pg.Pool({
  host: "localhost",
  port: 5432,
  database: "library",
  user: "app",
  password: process.env.PGPASSWORD,
});

async function getBooksByAuthor(authorId) {
  const sql = "SELECT title FROM books WHERE author_id = $1";
  const result = await pool.query(sql, [authorId]);
  return result.rows; // array of row objects
}
```

Why it matters: parameterised queries are the primary defence against SQL injection. The `$1` placeholder keeps user input as data, never as executable SQL.

Transactions in pg

When several statements must all succeed or all fail together (for example, transfer a book and log the move), run them on one client inside `BEGIN` / `COMMIT`, and `ROLLBACK` on error. A pooled `pool.query` may use a different connection each call, so check a single client out of the pool.

```
const client = await pool.connect();
try {
  await client.query("BEGIN");
  await client.query("UPDATE books SET author_id=$1 WHERE id=$2",
    [2, 10]);
  await client.query("INSERT INTO moves(book_id) VALUES($1)", [10]);
  await client.query("COMMIT");
} catch (err) {
  await client.query("ROLLBACK");
  throw err;
} finally {
  client.release(); // always return it to the pool
}
```

Gotcha: forgetting `client.release()` leaks connections until the pool is exhausted and every later query hangs.

MongoDB: documents and collections

MongoDB is a document database. A **document** is a BSON (binary JSON) object; documents live in **collections** (the loose equivalent of tables). There is no enforced schema by default, so two documents in one collection can have

different fields. Each document gets an `_id` (an `ObjectId`) automatically.

```
// a single document
{
  _id: ObjectId("..."),
  title: "Notes on Computing",
  tags: ["history", "math"],
  author: { name: "Ada Byron", email: "ada@example.com" }
}
```

Why it matters: related data can be **embedded** inside one document (the author object above), so a common read needs no join. The trade is duplication if that author appears in many books.

MongoDB CRUD (native driver)

The core methods are `insertOne`/`insertMany`, `find`/`findOne`, `updateOne`/`updateMany` (with update operators like `$set`), and `deleteOne`/`deleteMany`. `find` returns a cursor you turn into an array.

```
const books = db.collection("books");

await books.insertOne({ title: "Notes", tags: ["math"] });

const list = await books.find({ tags: "math" }).toArray();

await books.updateOne(
  { title: "Notes" },
  { $set: { published: 1843 } }
);

await books.deleteOne({ title: "Notes" });
```

Gotcha: `updateOne` without an operator like `$set` **replaces** the whole document with what you pass. Always use `$set` to change a field.

Mongoose: schemas and models

Mongoose is an ODM (object-document mapper) that adds a schema layer on top of MongoDB. A **Schema** defines fields, types, defaults, and validation; a **Model** is the constructor compiled from a schema and is your handle for querying a collection. Mongoose pluralises and lowercases the model name for the collection (`Book` -> `books`).

```
import mongoose from "mongoose";

const bookSchema = new mongoose.Schema({
  title:     { type: String, required: true, trim: true },
  pages:     { type: Number, min: 1 },
  published: { type: Date, default: Date.now },
});

const Book = mongoose.model("Book", bookSchema);

await mongoose.connect("mongodb://127.0.0.1:27017/library");
const b = await Book.create({ title: "Notes", pages: 200 });
```

Why it matters: the schema gives you validation and predictable shapes in an otherwise schemaless store, so the app can trust its documents.

Mongoose CRUD and queries

Model methods mirror CRUD: `create`, `find`/`findById`/`findOne`, `findByIdAndUpdate`, `findByIdAndDelete`. Queries are chainable and run when awaited.

```
const recent = await Book
  .find({ pages: { $gte: 100 } })
  .sort({ published: -1 })
  .limit(5);

await Book.findByIdAndUpdate(id, { pages: 250 },
  { new: true, runValidators: true });

await Book.findByIdAndDelete(id);
```

Gotcha: `findByIdAndUpdate` returns the **old** document by default. Pass `{ new: true }` to get the updated one, and `runValidators: true` because schema validation does not run on updates otherwise.

Mongoose relationships with refs

To model relationships across collections, store the related document's `_id` and mark it with `ref`. Then `populate` swaps the id for the full document at read time — Mongoose's join-like feature.

```
const bookSchema = new mongoose.Schema({
  title:  String,
  author: { type: mongoose.Schema.Types.ObjectId, ref: "Author" },
});

const withAuthor = await Book.findById(id).populate("author");
// withAuthor.author is now the full Author document
```

Why it matters: refs + populate keep data normalised (one source of truth for each author) at the cost of an extra query. Embedding is faster to read but duplicates data. Choose per access pattern.

REST: resources, verbs, status codes

REST models an API as **resources** addressed by URLs, acted on with HTTP verbs. Nouns go in the path, actions come from the method. Use plural resource names and put ids in the path, not the query string.

```
GET    /books        list books
POST   /books        create a book
GET    /books/42     read one book
PUT    /books/42     replace book 42
PATCH  /books/42     update part of book 42
DELETE /books/42     delete book 42
```

Status codes signal the outcome: 200 OK, 201 Created (with a `Location` header), 204 No Content (empty success, e.g. delete), 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error. Why it matters: correct codes let clients react without parsing the body.

Building a REST API in Express

Express maps method + path to handlers. Add `express.json()` to parse JSON request bodies. Read path params from `req.params`, query filters from `req.query`, and the parsed body from `req.body`. Send JSON with `res.status(code).json(...)`.

```
import express from "express";
const app = express();
app.use(express.json());

app.get("/books", async (req, res) => {
  const books = await Book.find();
  res.json(books);
});

app.post("/books", async (req, res) => {
  const book = await Book.create(req.body);
  res.status(201).location(`/books/${book.id}`).json(book);
});

app.get("/books/:id", async (req, res) => {
  const book = await Book.findById(req.params.id);
  if (!book) return res.status(404).json({ error: "not found" });
  res.json(book);
});

app.listen(3000);
```

Gotcha (Express 5, current as of 2026): a rejected promise in an `async` handler is now forwarded to error middleware automatically, so `await` failures no longer crash silently — but you should still add an error handler `app.use((err, req, res, next) => ...)` to return a clean 500 instead of a stack trace.

Cheatsheet

Task	SQL / pg	MongoDB / Mongoose
Create	`INSERT INTO t ...`	`Model.create({...})`
Read all	`SELECT * FROM t`	`Model.find()`
Read one	`... WHERE id=$1`	`Model.findById(id)`
Update	`UPDATE t SET ...`	`findByIdAndUpdate`
Delete	`DELETE FROM t ...`	`findByIdAndDelete`
Relate	`FOREIGN KEY` + JOIN	`ref` + `populate`
Run query	`pool.query(sql, [])`	`await Model.query`

HTTP verb	Purpose	Success code
GET	read	200
POST	create	201
PUT	replace	200
PATCH	partial update	200
DELETE	remove	204

pg / Mongoose bit	Meaning
`$1, $2`	pg parameter placeholders
`pool.connect()`	check out client (transactions)
`{ new: true }`	return updated doc
`runValidators: true`	validate on update
`$set`	change fields, not replace doc
`$gte / $lte / $in`	Mongo comparison operators

Common mistakes

- Building SQL by concatenating strings instead of using `$1` placeholders — this opens SQL injection.
- `UPDATE` or `DELETE` with no `WHERE`, changing every row.
- Using `pool.query` for a multi-statement transaction; each call may land on a different connection. Use a checked-out client.
- Forgetting `client.release()`, leaking pooled connections.
- Mongoose `updateOne`/replace without `$set`, wiping other fields.
- Expecting `findByIdAndUpdate` to return the new doc without `{ new: true }`, and to validate without `runValidators: true`.
- Putting verbs in REST paths (`/getBooks`) instead of using the HTTP method on a noun (`GET /books`).
- Returning 200 for everything; clients cannot tell created from found from failed.
- Forgetting `express.json()`, so `req.body` is `undefined`.

Quick self-check

1. What is the difference between a primary key and a foreign key?
2. Which join returns authors that have no books, and how?

3. Why pass query values as `[value]` with `$1` instead of inline?
4. When must you use a checked-out client instead of `pool.query`?
5. In MongoDB, what does `updateOne` do if you omit `$set`?
6. What does Mongoose `populate` do, and what is its cost?
7. Which two options does `findByIdAndUpdate` usually need, and why?
8. Which HTTP verb and status code fit "create a new book"?
9. When is 204 the right status code?
10. What middleware makes `req.body` available for JSON requests?

Answers

1. A primary key uniquely identifies a row in its own table; a foreign key stores another table's primary key to link the two.
2. A `LEFT JOIN` from authors to books with `WHERE books.id IS NULL` — authors kept, unmatched book side is NULL.
3. The placeholder keeps input as data, so `pg` escapes it and SQL injection is prevented; concatenation runs user input as SQL.
4. For a transaction (`BEGIN`/`COMMIT`/`ROLLBACK`), because all statements must run on the same connection.
5. It replaces the entire document with what you pass, dropping any fields not included.
6. It replaces stored `ObjectId` refs with the full referenced documents; the cost is one extra query per populated path.
7. `{ new: true }` to return the updated document and `runValidators: true` so schema rules run on the update.
8. `POST /books` returning 201 Created (ideally with a `Location` header pointing at the new resource).
9. On a successful request with no body to return, such as a `DELETE` or an update that sends nothing back.
10. `express.json()`, registered with `app.use()` before the routes.

Chapter 5: Authentication and React

What this covers

This chapter maps to the authentication and React sections of the bootcamp. The first half covers how to store passwords safely with hashing, the difference between server-side sessions and JSON Web Tokens, wiring up Passport.js with a local username/password strategy, keeping secrets out of source control, and a high-level look at OAuth "Log in with Google". The second half switches to the front end: React 18 fundamentals — JSX, components, props, state with `useState`, side effects with `useEffect`, event handling, rendering lists with keys, controlled forms, conditional rendering, and why modern React uses hooks instead of class components.

Key concepts

Never store plaintext passwords — hash with bcrypt

A password database should never contain the actual passwords. If it leaks, every account is compromised. Instead you store a one-way hash. bcrypt is a slow, salted hashing algorithm designed exactly for this: the salt defends against rainbow tables, and the deliberate slowness defends against brute force. The "cost factor" (salt rounds) controls how slow; 10-12 is a sensible default as of 2026.

```
import bcrypt from "bcrypt";

const SALT_ROUNDS = 12;

// On registration: hash before saving
const hash = await bcrypt.hash(plainPassword, SALT_ROUNDS);
await User.create({ email, password: hash });

// On login: compare, never decrypt (bcrypt is one-way)
const user = await User.findOne({ email });
const ok = user && (await bcrypt.compare(plainPassword, user.password));
```

Why it matters: `bcrypt.compare` re-hashes the input with the stored salt and checks in constant time. You never reverse a hash — there is no `bcrypt.decrypt`.

Sessions vs JWT — two ways to stay logged in

HTTP is stateless, so after login you need a way to remember who the user is on the next request. Two common approaches:

- **Sessions (stateful):** the server stores session data (e.g. in memory, Redis,

or Mongo) and sends the client a signed cookie holding only a session ID. Each request looks the ID up server-side. Easy to revoke (delete the record), but the server must keep state.

- **JWT (stateless):** the server signs a token containing the user's claims and

sends it to the client. The client returns it on each request; the server verifies the signature and trusts the payload without a lookup. Scales well, but hard to revoke before expiry.

```
import jwt from "jsonwebtoken";

// Issue a token at login
const token = jwt.sign({ sub: user.id }, process.env.JWT_SECRET, {
  expiresIn: "1h",
});

// Verify on a protected route
const payload = jwt.verify(token, process.env.JWT_SECRET); // throws if invalid
```

Gotcha: a JWT payload is only base64-encoded, not encrypted — anyone can read it. Never put secrets in it, and always check the signature before trusting it. For revocation, keep tokens short-lived and pair them with a refresh token.

Passport.js local strategy

Passport is authentication middleware for Express. A "strategy" is a plug-in for one login method; the local strategy handles username/password. You give Passport a verify callback, and it calls `done(err, user)` with the result. Passport then manages the logged-in user, typically on the session.

```
import passport from "passport";
import { Strategy as LocalStrategy } from "passport-local";

passport.use(
  new LocalStrategy({ usernameField: "email" }, async (email, pw, done) => {
    try {
      const user = await User.findOne({ email });
      if (!user) return done(null, false, { message: "No user" });
      const ok = await bcrypt.compare(pw, user.password);
      return ok ? done(null, user) : done(null, false);
    } catch (err) {
      return done(err);
    }
  })
);

// Store user id in the session, and reload the user from it
passport.serializeUser((user, done) => done(null, user.id));
passport.deserializeUser(async (id, done) => {
  done(null, await User.findById(id));
});
```

Why it matters: `serializeUser` decides what goes into the session (keep it small — just the id), and `deserializeUser` rehydrates the full user on each request. Wire it up with `app.use(passport.initialize())` and `app.use(passport.session())` after your session middleware.

Environment secrets — keep keys out of code

Secrets (DB passwords, `JWT_SECRET`, OAuth client secrets, API keys) must never be committed. Put them in a `.env` file, load them into `process.env`, and add `.env` to `.gitignore`. Commit a `.env.example` with the keys but no values so teammates know what to set.

```
import "dotenv/config"; // loads .env into process.env at startup

const secret = process.env.JWT_SECRET;
if (!secret) throw new Error("JWT_SECRET is not set");
```

```
# .env  (never commit this)
JWT_SECRET=some-long-random-string
GOOGLE_CLIENT_SECRET=xxxx
```

Gotcha: anything already pushed to git stays in history even after you delete it. If a secret leaks, rotate it — don't just remove the line.

OAuth and "Log in with Google" (high level)

OAuth 2.0 lets a user log in with an existing account (Google, GitHub, etc.) without giving you their password. At a high level the "authorization code" flow is:

1. Your app redirects the user to Google with your client ID and requested scopes.
2. The user authenticates with Google and consents.
3. Google redirects back to your callback URL with a one-time **code**.
4. Your server exchanges the code (plus your client secret) for tokens and the

user's profile, then creates or looks up a local account.

```
import { Strategy as GoogleStrategy } from "passport-google-oauth20";

passport.use(
  new GoogleStrategy(
    {
      clientID: process.env.GOOGLE_CLIENT_ID,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
      callbackURL: "/auth/google/callback",
    },
```

```
    async (accessToken, refreshToken, profile, done) => {
      const user = await User.findOrCreateByGoogle(profile);
      done(null, user);
    }
  )
);
```

Why it matters: the client secret and the code exchange happen server-side, never in the browser. Users with Google login have no password row at all, so your schema should allow accounts without a password hash.

JSX and components

React UI is built from **components** — functions that return markup. The markup is **JSX**, which looks like HTML but is really JavaScript: it compiles to `React.createElement` calls. A component name must be capitalized, and each component returns a single root element (use a `<>...</>` fragment to avoid an extra wrapper).

```
function Greeting() {
  const name = "Rizwan";
  return (
    <>
      <h1>Hello, {name}</h1>
      <p>Welcome back.</p>
    </>
  );
}
```

Gotcha: JSX uses `className` instead of `class` and `htmlFor` instead of `for`, because `class`/`for` are reserved words in JavaScript. Curly braces `{ }` embed any JS expression.

Props — passing data down

Props are the inputs to a component, passed like HTML attributes. They are read-only: a component must never modify its own props. Data flows one way, from parent to child.

```
function UserCard({ name, role }) {
  return <div className="card">{name} - {role}</div>;
}

function App() {
  return <UserCard name="Rizwan" role="Full-stack dev" />;
}
```

Why it matters: `children` is a special prop holding whatever you nest between a component's tags, which is how you build wrappers and layouts.

State and useState

State is data a component owns and can change over time; changing it re-renders the component. In function components you declare state with the `useState` hook, which returns the current value and a setter.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>Clicked {count}</button>;
}
```

Gotcha: never mutate state directly (`count++`) — always call the setter. When the next value depends on the previous one, use the functional form `setCount(c => c + 1)` to avoid stale values, especially inside async code or multiple updates in one handler.

useEffect — side effects

`useEffect` runs code after render for things outside React's render flow: data fetching, subscriptions, timers, manual DOM work. The dependency array controls when it re-runs, and an optional returned function cleans up.

```
import { useEffect, useState } from "react";

function Profile({ userId }) {
  const [user, setUser] = useState(null);
```

```
  useEffect(() => {
    let active = true;
    fetch(`/api/users/${userId}`)
      .then((r) => r.json())
      .then((data) => active && setUser(data));
    return () => {
      active = false; // cleanup: ignore result if unmounted/changed
    };
  }, [userId]); // re-run only when userId changes

  return <p>{user ? user.name : "Loading..."}</p>;
}
```

Gotcha: `[]` runs the effect once on mount; omitting the array runs it after every render (usually a bug). List every value from the component that the effect uses, or you get stale data. In React 18 Strict Mode (dev only), effects run twice on mount to surface missing cleanup — that is expected.

Event handling

React events use camelCase names (`onClick`, `onChange`, `onSubmit`) and take a function, not a string. You usually pass an inline arrow or a named handler. The event object is a synthetic wrapper over the native one.

```
function SearchBox() {
  function handleSubmit(e) {
    e.preventDefault(); // stop the browser's full-page reload
    console.log("searching...");
  }
  return <form onSubmit={handleSubmit}><button>Go</button></form>;
}
```

Why it matters: pass `onClick={handleClick}`, not `onClick={handleClick()}` — the second calls the function during render instead of on the click.

Lists and keys

Render a collection by mapping an array to elements. Each sibling element needs a stable, unique `key` so React can track items across re-renders efficiently.

```
function TodoList({ todos }) {
  return (
    <ul>
      {todos.map((t) => (
        <li key={t.id}>{t.title}</li>
      ))}
    </ul>
  );
}
```

Gotcha: don't use the array index as a key if the list can reorder, filter, or have items inserted — it causes wrong state to stick to the wrong row. Use a stable id from the data.

Controlled forms

In a **controlled** input, React state is the single source of truth: the input's `value` comes from state and `onChange` updates it. This lets you validate, format, or disable in real time.

```
function LoginForm() {
  const [email, setEmail] = useState("");

  return (
    <input
      type="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
    />
  );
}
```

Why it matters: an input with `value` but no `onChange` is read-only and warns in the console. For several fields, keep an object in state and update by key with a spread: `setForm({ ...form, [e.target.name]: e.target.value })`.

Conditional rendering

Show different UI based on state using plain JavaScript: a ternary, `&&`, or an early `return`. There is no special template syntax.

```
function Status({ user }) {
  return (
    <div>
      {user ? <span>Hi {user.name}</span> : <a href="/login">Log in</a>}
      {user?.isAdmin && <button>Admin panel</button>}
    </div>
  );
}
```

Gotcha: with `&&`, a falsy left side that is `0` renders the literal `0`. Guard with a real boolean (`count > 0 && ...`) rather than `count && ...`.

Hooks vs class components

React used to require class components (with `this.state`, `setState`, and lifecycle methods like `componentDidMount`) for anything stateful. Since React 16.8, **hooks** let function components do everything classes did, with less boilerplate and easier logic reuse via custom hooks. As of 2026, function components with hooks are the standard; write new code this way and reserve classes for legacy code or error boundaries.

```
// Modern: function + hooks (preferred)
function Timer() {
  const [n, setN] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setN((x) => x + 1), 1000);
    return () => clearInterval(id);
  }, []);
  return <p>{n}s</p>;
}
```

Why it matters: hooks can only be called at the top level of a component (not inside loops, conditions, or nested functions) and only from React functions. That "rules of hooks" constraint is what keeps their order stable across renders.

Cheatsheet

Task	Syntax / API
Hash password	`await bcrypt.hash(pw, 12)`
Check password	`await bcrypt.compare(pw, hash)`
Sign JWT	`jwt.sign(payload, secret, {expiresIn})`
Verify JWT	`jwt.verify(token, secret)`
Load env vars	`import "dotenv/config"`
Passport local	`new LocalStrategy(opts, verifyCb)`
Save to session	`passport.serializeUser(cb)`
Protect route	`req.isAuthenticated()`
Local state	`const [v, setV] = useState(init)`
Update from prev	`setV(prev => prev + 1)`
Side effect	`useEffect(fn, [deps])`
Effect cleanup	`return () => {...}` inside effect
Handle click	`onClick={handler}`
Prevent reload	`e.preventDefault()`
Render list	`arr.map(x => <li key={x.id}>...)`
Controlled input	`value={v} onChange={...}`
Conditional	`{cond ? <A/> : <B/>}` or `{cond && <A/>}`

Task	Syntax / API
Fragment	`<>...</>`

Common mistakes

- Storing plaintext or reversible passwords instead of a bcrypt hash.
- Trusting a JWT payload without verifying its signature, or putting secrets in it.
- Committing a `.env` file, then thinking `git rm` erases it from history.
- Forgetting `passport.session()` after the session middleware, so login doesn't persist.
- Handling the OAuth code exchange or client secret in the browser instead of the server.
- Mutating React state directly (`arr.push(...)`, `count++`) instead of using the setter.
- Leaving out the `useEffect` dependency array, causing an effect to run on every render.
- Calling a handler during render: `onClick={doThing()}` instead of `onClick={doThing}`.
- Using the array index as a `key` in a list that reorders or filters.
- A controlled input with `value` but no `onChange` (read-only warning).
- Calling a hook inside a condition or loop, breaking the rules of hooks.

Quick self-check

1. Why can't you "decrypt" a bcrypt hash to check a password?
2. What is the main trade-off between server sessions and JWTs?
3. In Passport's local strategy, what do you pass to `done` on a wrong password?
4. What does `serializeUser` decide?
5. If a secret was committed and pushed, is deleting the line enough?
6. Why is `className` used in JSX instead of `class`?
7. What does the `useState` setter do besides changing the value?
8. What does an empty dependency array `[]` mean in `useEffect`?
9. Why should a list `key` be stable rather than the array index?
10. Name one constraint from the rules of hooks.

Answers

1. bcrypt is one-way; there is no decrypt. `bcrypt.compare` re-hashes the input with the stored salt and compares the results.
2. Sessions keep state on the server (easy to revoke, needs storage); JWTs are stateless (scale well, hard to revoke before expiry).
3. `done(null, false)` — no error, but no authenticated user (optionally a

message object as the third argument).

4. What gets stored in the session — keep it minimal, usually just the user's id.
5. No. It stays in git history; you must rotate (change) the secret.
6. `class` is a reserved word in JavaScript, so JSX uses `className`.
7. It triggers a re-render of the component with the new state.
8. Run the effect once after the first render (on mount), and never again.
9. A stable key ties state to the right item when the list reorders or filters;

an index shifts and attaches state to the wrong row.

10. Only call hooks at the top level of a React function, never inside loops,

conditions, or nested functions.

Chapter 6: Web3, the Internet Computer and DApps

What this covers

This chapter revises the final block of the bootcamp (course Sections 37-41): what Web3 and a decentralised app (DApp) are, the Internet Computer Protocol (ICP) and its canister smart contracts, the DFINITY SDK and the `dfx` command-line tool, the basics of the Motoko language, building a first DeFi DApp in the style of the course "DBANK" (a canister that stores a balance, tops up, withdraws, and pays compound interest on a timer), deploying to the ICP mainnet at a high level (cycles, wallet, identity), wiring a React frontend to a canister with the generated actor and `@dfinity/agent`, and creating a simple crypto token canister. The tooling moves fast, so every command and syntax note here is checked against the official docs at internetcomputer.org and marked "as of 2026" where it has changed since the course was recorded.

Important framing before you start: the course was recorded on an older `dfx` and older Motoko. Several things changed. The three big ones are (1) Motoko actors are now "persistent" by default with enhanced orthogonal persistence, (2) the old `system func heartbeat` is superseded by the Timer library, and (3) the old cycles wallet is being replaced by the cycles ledger and the `dfx cycles` command. Each is flagged in place below so your revision matches current reality, not the 2021 videos.

Key concepts

Web3 in one paragraph

Web1 was read-only pages, Web2 is read-write apps owned by companies (your data and logic live on their servers), and Web3 is read-write-own: application logic and state run on a public, decentralised network rather than one company's cloud. In Web3 there is no single server you rent; code is deployed to many independent nodes that reach consensus, and users interact with cryptographic identities instead of email-and-password accounts. The Internet Computer is one such network. Why it matters: the same JavaScript frontend skills transfer, but the "backend" is now a smart contract you cannot silently change or take down.

What a DApp is

A DApp (decentralised application) is an app whose backend logic and data live in smart contracts on a blockchain, usually with a normal web frontend on top. The frontend can also be served from the chain, so the whole thing runs without a traditional server.

```
Browser (React)  ->  Agent (JS)  ->  Canister smart contract (on ICP)
   UI / state        signs calls       logic + persisted state
```

Contrast with a Web2 app: instead of `fetch('/api/balance')` hitting your Express server, the frontend calls a canister method directly over the IC's protocol. Gotcha: "decentralised" does not mean "no rules" - you still design an API, you just cannot patch it as casually, and every state-changing call costs resources (cycles).

The Internet Computer Protocol (ICP) and canisters

The Internet Computer is a blockchain network run by independent node providers, organised into subnets, that hosts code and data directly (see internetcomputer.org). Its unit of deployment is the canister: a smart contract that bundles a WebAssembly (Wasm) module with its own persistent memory. A canister is like a tiny always-on server with a Candid interface (its typed API), except it runs on-chain, keeps its state across upgrades, and is metered in cycles. You can write canisters in Motoko (ICP's native language), Rust, or any language that compiles to Wasm; the course uses Motoko.

Key properties to remember:

- Canisters communicate by messages; a call is either an update (changes state, goes

through consensus, slower) or a query (read-only, fast, not consensus-verified).

- Each canister has a principal (its on-chain identity/address).

- Canisters are paid for by cycles, not gas paid by the caller - this is the "reverse

gas" model, so end users can call your DApp for free.

The DFINITY SDK and the `dfx` tool

`dfx` is the command-line tool for building, deploying, and managing canisters. As of 2026 you install the IC SDK with the official one-line script from internetcomputer.org (the older `sh -ci "$(curl ...install.sh)"` DFINITY install command), then `dfx --version` to confirm; current releases are in the 0.28+ range with Motoko compiler 0.15+.

Core loop: create a project, start a local replica, deploy, then call methods.

```
dfx new dbank              # scaffold a project (prompts: language + frontend)
cd dbank
dfx start --background     # run a local replica in the background
dfx deploy                 # create + build + install all canisters locally
dfx canister call dbank topUp   # call a method on the dbank canister
dfx stop                   # shut the local replica down
```

As of 2026 note: `dfx new` now prompts you to choose a backend language (Motoko/Rust) and a frontend (React, Svelte, Vue, plain JS, or none). `dfx start` uses PocketIC as the local execution environment (you may see or pass `--pocketic`); older courses show a different local replica but the commands you type are the same. `dfx deploy` is still the one command that runs create, build, and install together.

Candid and `dfx generate`

Candid is the IC's interface description language: it describes a canister's methods and types in a language-neutral way (a `.did` file) so any client - a Rust canister, a JS frontend, the `dfx canister call` CLI - can talk to it. From your Motoko actor, `dfx` produces the Candid automatically. `dfx generate` then produces JavaScript/TypeScript "declarations" (an `idlFactory` plus a `createActor` helper) that the frontend imports. Why it matters: you rarely hand-write the wire format; you write Motoko types and the tooling generates the typed client for you.

Motoko basics: actors

A Motoko program that becomes a canister is written as an actor. An actor encapsulates private state and exposes public async methods; only one message runs at a time, so you do not manage locks. As of 2026 you declare it `persistent actor`, which turns on enhanced orthogonal persistence: the actor's variables keep their values across upgrades by default, so you no longer have to mark everything `stable` by hand.

```
import Debug "mo:base/Debug";

persistent actor Counter {
  var count : Nat = 0;        // persists across upgrades (persistent actor)

  public func increment() : async Nat {
    count += 1;
    count
  };

  public query func get() : async Nat {
    count
  };
}
```

As of 2026 change: in the course you will see a plain `actor { ... }` with `stable var currentValue`. That still compiles, but the modern idiom is `persistent actor`, where ordinary `var` bindings are already persistent and the `flexible` keyword (its opposite) has been removed along with the old "classical" persistence. See internetcomputer.org data-persistence docs.

Public, query, and update functions

Public functions form the canister's API and must return `async T`. A plain public function is an update call: it can change state and its result is agreed by consensus. A `public query func` is read-only and fast, but its result is not consensus-verified, so never trust a query for anything security-critical (like confirming a payment).

```
public func deposit(amount : Nat) : async () { balance += amount };  // update
public query func balanceOf() : async Nat { balance };              // query
```

Gotcha: a `query` cannot call an update method or perform inter-canister update calls, and its state changes are discarded when it returns. Use update for writes, query for cheap reads.

Motoko types and stable variables

Motoko is strongly, statically typed. Common types: `Nat` (unbounded non-negative int), `Int`, `Float`, `Bool`, `Text`, `Char`, tuples, `Blob`, `Principal`, plus records `{ name : Text }`, variants `{ #ok; #err }`, arrays `[Nat]`, and `Option ?T`. A "stable variable" is one whose value survives a canister upgrade (new code, same state).

As of 2026: with `persistent actor` every `var` is stable-by-default, so you mostly do not write the `stable` keyword. You still care about "stable compatibility" on upgrade: if a variable's type changes incompatibly the upgrade traps. The tooling warns you, and Motoko supports explicit data migration when you need to reshape state (see the compatibility docs on internetcomputer.org).

```
persistent actor Store {
  var owner : Text = "rizwan";     // kept across upgrades automatically
  transient var cache : Nat = 0;   // 'transient' = reset on upgrade (opt out)
}
```

The base vs core standard library

The course imports from `mo:base` (for example `mo:base/Debug`, `mo:base/Float`). As of 2026 there is a new standard library, `mo:core`, that is replacing `base`; new projects are encouraged to use it (needs dfx 0.28+/moc 0.15+, added via the Mops package manager). The APIs are similar but not identical - notably `range()` functions in `core` are exclusive of the upper bound where the old `Iter.range` was inclusive. For revision, recognise both: examples here use `mo:base` to match the course, but if you start fresh today prefer `mo:core` and check the "base to core migration guide" on internetcomputer.org.

Building the DeFi DApp (DBANK style)

The course's DBANK is a canister that holds a balance, lets you top up and withdraw, and periodically pays compound interest. Here is an original, minimal version using modern syntax. It stores a single account balance as a `Float`.

```
import Debug "mo:base/Debug";
import Float "mo:base/Float";
import Timer "mo:base/Timer";

persistent actor DBank {
  var balance : Float = 300.0;
  let rate : Float = 0.01;   // 1% interest per period

  public func topUp(amount : Float) : async () {
    balance += amount;
    Debug.print("Topped up. Balance = " # Float.toText(balance));
  };

  public func withdraw(amount : Float) : async () {
    if (amount <= balance) {
      balance -= amount;
    } else {
      Debug.print("Insufficient funds");
    };
  };

  public query func checkBalance() : async Float {
    balance
  };

  // Pay compound interest once per period.
  func payInterest() : async () {
    balance += balance * rate;
  };
}
```

Why `Float`: the course uses it for simplicity so you can see decimals. In real DeFi you use integer base units (like `Nat`, in the smallest denomination) to avoid rounding errors - floats are fine for learning, not for money.

Timers instead of heartbeat

The course schedules interest with `system func heartbeat`, which the runtime called on a fixed, non-adjustable interval. As of 2026 the recommended mechanism is the Timer library: `Timer.setTimer` for a one-off and `Timer.recurringTimer` for a repeating job, each returning a `TimerId` you can `cancelTimer`. Heartbeat still exists but timers are cheaper and let you pick the interval (see the periodic-tasks docs on internetcomputer.org).

```
import Timer "mo:base/Timer";

persistent actor DBank {
  var balance : Float = 300.0;

  func payInterest() : async () { balance += balance * 0.01 };

  // Start a recurring timer when the canister is first installed.
  ignore Timer.recurringTimer<system>(
    #seconds 3600,        // every hour
    payInterest
  );
}
```

Gotcha: a timer set at the top level of the actor body starts on install; after an upgrade you typically re-arm timers in a post-upgrade hook, because pending timers do not automatically survive a code upgrade. Keep the interval realistic - every call burns cycles, so "every second" on mainnet is expensive.

Deploying to the ICP mainnet (high level)

Local deploys are free; mainnet costs cycles, and cycles come from ICP tokens. As of 2026 the workflow is:

1. Create an identity: `dfx identity new me` then `dfx identity use me`. Your identity is

a keypair with a principal - the on-chain "who" for your deploys.

2. Get ICP into the account for that identity (buy on an exchange, send to your ledger

account: `dfx ledger account-id`).

3. Convert ICP to cycles on the cycles ledger: `dfx cycles convert --amount 2 --network ic`.

Cycles are the "fuel" (stable in value, unlike a volatile gas token).

4. Deploy: `dfx deploy --network ic`. New canisters are funded from your cycles balance.
5. Top up a running canister later with `dfx cycles top-up <amount> <canister> --network ic`.

As of 2026 change: the course uses a cycles wallet (a helper canister that held your cycles, created via `dfx ledger create-canister` + `dfx identity deploy-wallet`). That still works, but the cycles ledger (default since dfx 0.22) replaces it, and the cycles wallet is slated for removal - so learn `dfx cycles ...`. If a wallet is configured dfx uses it, otherwise it uses the cycles ledger. Also watch the "freezing threshold": a canister that runs out of cycles is uninstalled and its state is lost, so keep it topped up.

Wiring a React frontend to a canister

After `dfx deploy` (or `dfx generate`) you get generated declarations under `src/declarations/<canister>`. The frontend imports the ready-made `createActor` and the `canisterId`, and calls canister methods like async functions. Install `@dfinity/agent` (the low-level agent that encodes/decodes calls) if you construct the actor by hand.

```
// Simple path: use the generated helpers.
import { dbank } from "../declarations/dbank";

async function refresh() {
  const balance = await dbank.checkBalance();  // a query call
  document.getElementById("bal").innerText = balance;
}

async function topUp(amount) {
  await dbank.topUp(Number(amount));           // an update call
  await refresh();
}
```

If you need to control the agent (for example to point at the local replica or attach an identity) you build the actor yourself:

```
import { Actor, HttpAgent } from "@dfinity/agent";
import { idlFactory } from "../declarations/dbank";

const agent = await HttpAgent.create({ host: "http://127.0.0.1:4943" });
// Local only: trust the replica's root key. Never do this against mainnet.
await agent.fetchRootKey();

const dbank = Actor.createActor(idlFactory, {
  agent,
  canisterId: process.env.CANISTER_ID_DBANK,
});
```

Gotchas: (1) values cross the boundary as Candid, so a Motoko `Nat` may arrive as a JavaScript `BigInt` - convert before doing math or rendering. (2) `fetchRootKey()` is for local development only; calling it against mainnet is a security hole. (3) In React, create the agent/actor inside `useEffect` (or a lazy import) so it does not run during server-side rendering.

Creating a simple crypto token canister

A token is just a canister that maps owners to balances and exposes transfer and balance methods. Here is a stripped-down educational token (not a full standard):

```
import Principal "mo:base/Principal";
import Map "mo:base/OrderedMap";

persistent actor Token {
  let ops = Map.Make<Principal>(Principal.compare);
  var balances = ops.empty<Nat>();
  let owner = Principal.fromText("aaaaa-aa");  // set to your principal

  public shared (msg) func transfer(to : Principal, amt : Nat) : async Bool {
    let from = msg.caller;
    let bal = switch (ops.get(balances, from)) { case null 0; case (?b) b };
    if (bal < amt) return false;
    balances := ops.put(balances, from, bal - amt);
    let toBal = switch (ops.get(balances, to)) { case null 0; case (?b) b };
    balances := ops.put(balances, to, toBal + amt);
    true
  };

  public query func balanceOf(p : Principal) : async Nat {
    switch (ops.get(balances, p)) { case null 0; case (?b) b }
  };
}
```

Note `shared (msg)`: it exposes the caller's principal via `msg.caller`, which is how the canister knows who is spending. As of 2026, for anything real you should implement the ICRC-1 standard (the IC's fungible-token standard: transfer, balance, and metadata methods, with accounts as principal + subaccount), rather than a bespoke interface, so wallets and exchanges can talk to your token. The course's custom token is a great teaching model; ICRC-1 is what you ship. See the digital-asset-standards docs on internetcomputer.org.

Cheatsheet

Command / syntax	What it does
`dfx new <name>`	Scaffold a project (pick lang + frontend)
`dfx start --background`	Run local replica (PocketIC) in background
`dfx stop`	Stop the local replica
`dfx deploy`	Create + build + install all canisters (local)
`dfx deploy --network ic`	Deploy to mainnet
`dfx canister call <c> <m>`	Call method `m` on canister `c`
`dfx generate`	Emit JS/TS declarations for frontends

Command / syntax	What it does
`dfx identity new/use <id>`	Create / switch identity (keypair)
`dfx cycles convert --amount N`	Turn ICP into cycles (cycles ledger)
`dfx cycles top-up N <canister>`	Add cycles to a running canister
`dfx ledger account-id`	Show your ICP ledger account
`persistent actor { ... }`	Canister with persist-by-default state
`var x : Nat = 0;`	Persistent field (in persistent actor)
`transient var x`	Field reset on upgrade (opt out)
`public func f() : async T`	Update method (writes, consensus)
`public query func f() : async T`	Read-only fast method
`shared (msg) func f()`	Exposes `msg.caller` (Principal)
`Timer.recurringTimer<system>(d, job)`	Repeating scheduled task
`Timer.setTimer<system>(d, job)`	One-off scheduled task
`mo:base/...` / `mo:core/...`	Old / new standard library imports

Concept	One-liner
Canister	Wasm module + persistent memory = smart contract
Principal	On-chain identity/address of caller or canister
Cycles	Fuel a canister spends; from converting ICP
Reverse gas	Canister pays, so end users call for free
Candid (.did)	Language-neutral interface description
Update call	Changes state, consensus-verified, slower
Query call	Read-only, fast, not consensus-verified
ICRC-1	Standard fungible-token interface on ICP

Common mistakes

- Trusting a `query` result for anything security-sensitive - queries are not consensus-verified; use an update call to confirm payments or balances.
- Calling `agent.fetchRootKey()` against mainnet. It is local-development only and disabling that check on mainnet is a serious security hole.
- Forgetting a `Nat`/`Int` arrives in JavaScript as `BigInt`; doing math or template rendering without converting throws or shows the wrong value.
- Using `Float` for real money. It is fine for the DBANK demo but real tokens use integer base units to avoid rounding drift.
- Expecting `system func heartbeat` to be the modern way - as of 2026 prefer the Timer library, and re-arm timers after an upgrade because pending timers do not survive it.
- Assuming state is safe when cycles run out. A canister below its freezing threshold is uninstalled and loses state; keep it topped up.
- Copying old course syntax blindly: `flexible` is removed, actors are `persistent` by default, and the cycles wallet is being replaced by the cycles ledger (`dfx cycles`).
- Changing a stable variable's type incompatibly across an upgrade - the upgrade traps unless you provide a proper migration.

- Importing generated declarations from the `.dfx` folder in a React app. Import from the generated `src/declarations/...` path instead, and build the actor inside `useEffect`.

Quick self-check

1. What is the difference between an update call and a query call on a canister?
2. What are cycles, and how do you get them?
3. In modern Motoko, what does `persistent actor` change compared to the course's plain `actor` with `stable var`?
4. Why should you not use `Float` for a production token balance?
5. What does `shared (msg)` give you inside a public function, and why is it needed for a token transfer?
6. Which command turns your Motoko interface into a JS/TS client for the frontend?
7. What replaced `system func heartbeat` for scheduling periodic work, as of 2026?
8. What is `agent.fetchRootKey()` for, and when must you never call it?
9. What is Candid, and what file represents it?
10. What standard should a real fungible token implement on ICP, and why?

Answers

1. An update call can change state and is agreed by consensus (slower, verifiable); a query is read-only, fast, and not consensus-verified, so its result is not trustworthy for security-critical decisions.
2. Cycles are the resource a canister spends to run and store data ("fuel"). You get them by converting ICP tokens to cycles on the cycles ledger, for example `dfx cycles convert --amount 2 --network ic`.
3. With `persistent actor`, enhanced orthogonal persistence is on by default, so ordinary `var` fields persist across upgrades without the `stable` keyword; the opposite keyword `flexible` has been removed and you use `transient` to opt a field out.
4. Floating-point arithmetic introduces rounding errors; money should be tracked in integer base units (smallest denomination) so balances stay exact.
5. `shared (msg)` exposes `msg.caller`, the Principal of whoever called the method. A transfer needs to know who is sending so it can debit the correct account.
6. `dfx generate` (it produces the declarations with `idlFactory` and `createActor`; `dfx deploy` also generates them).
7. The Timer library: `Timer.setTimer` for one-off and `Timer.recurringTimer` for repeating jobs, each returning a cancellable `TimerId`.
8. It fetches the local replica's root key so the agent can verify responses during local development. Never call it against mainnet - it disables a real signature check.
9. Candid is the IC's language-neutral interface description language; a canister's interface is described in a `.did` file, generated from your Motoko types.
10. ICRC-1 (the IC fungible-token standard). Implementing it lets wallets, exchanges, and other canisters interact with your token through a known, common interface.

Chapter 7: Web Design Theory

What this covers

This chapter revises the Web Design School part of the bootcamp: the design principles a developer needs to make a site look intentional rather than default. It covers colour theory (the wheel, common schemes, contrast and accessibility, and the tools to check it), typography (serif vs sans-serif, pairing fonts, hierarchy, and readable line length and height), User Interface (UI) design principles (whitespace, alignment, hierarchy, consistency, and Fitts's law), User Experience (UX) design principles (simplicity, feedback, affordances, and user flows), and layout (grid systems, the classic 960 grid, and visual hierarchy). The aim is to give you a checklist of reasons for the CSS you already know how to write.

Key concepts

The colour wheel

The colour wheel arranges hues in a circle so you can reason about their relationships by position. The three primary colours (red, yellow, blue in the traditional artist's model) mix into secondaries (orange, green, purple), and those mix into tertiaries. Position matters more than the exact hue: colours near each other feel calm and related, colours opposite each other feel high-energy and contrasting.

Three properties describe any single colour, and CSS `hsl()` maps directly onto them, which is why it is the most designer-friendly colour format:

```
.brand {
  /* hue 210 (a blue), 80% saturation, 50% lightness */
  color: hsl(210, 80%, 50%);
}
.brand-muted { color: hsl(210, 30%, 50%); }  /* less saturated */
.brand-dark  { color: hsl(210, 80%, 30%); }  /* darker */
```

Why it matters: with `hsl()` you build a whole palette by holding the hue fixed and only changing saturation and lightness, instead of guessing hex codes.

Colour schemes

A scheme is a rule for picking hues off the wheel so they work together. Learn a few and you can generate a palette on demand:

- **Monochromatic**: one hue, varying lightness and saturation. Safe,

cohesive, low effort.

- **Analogous**: 2-3 neighbouring hues (e.g. blue, blue-green, green).

Harmonious, good for calm designs.

- **Complementary**: two opposite hues (e.g. blue and orange). Strong

contrast, ideal for a call-to-action against a background.

- **Triadic**: three hues evenly spaced (120 apart). Vibrant but

balanced.

```
:root {
  --primary:   hsl(210, 80%, 50%);  /* blue */
  --accent:    hsl(30, 90%, 55%);   /* complementary orange */
  --neutral:   hsl(210, 15%, 25%);  /* dark, low-sat text colour */
}
```

Gotcha: use one dominant colour, one accent for actions, and neutrals (near-greys) for most of the page. The common 60-30-10 guideline is 60% dominant, 30% secondary, 10% accent.

Contrast and accessibility

Contrast is the measured difference in luminance between two colours, usually text and its background. The Web Content Accessibility Guidelines (WCAG) set minimum contrast ratios so text stays readable for low-vision users: **4.5:1** for normal body text (AA), **3:1** for large text (roughly 24px, or 18.7px bold) and for UI components like input borders. The stricter AAA level asks for 7:1.

```
/* Passes AA: dark text on light bg is about 12:1 */
.body-text { color: hsl(210, 20%, 20%); background: hsl(0, 0%, 100%); }

/* Fails AA: light grey on white is often ~2:1 */
.too-faint { color: #bbbbbb; background: #ffffff; }
```

Why it matters: never signal meaning by colour alone (e.g. a red-only error). Add an icon or text, so colour-blind users and anyone in bright sunlight still get the message.

Colour tools

You do not judge contrast or harmony by eye. Common tools, as of 2026:

- **Adobe Color** (color.adobe.com) and **Coolors** (coolors.co) to

generate schemes from the wheel or from a photo.

- The **WebAIM Contrast Checker** (webaim.org/resources/contrastchecker)

for a pass/fail against WCAG ratios.

- Browser **DevTools**: the colour picker in the Styles panel shows the

contrast ratio and an AA/AAA badge inline, and the Lighthouse audit flags low-contrast text automatically.

```
Workflow: pick hue in Coolors -> lock the accent ->
check text pairs in WebAIM -> confirm in DevTools contrast picker.
```

Gotcha: check contrast on the actual overlay colours, including text on images and on semi-transparent overlays, not just your named tokens.

Serif vs sans-serif

Serifs are the small feet on the ends of letter strokes. **Serif** faces (Georgia, Times, Merriweather) feel traditional, formal, editorial, and read well in long print passages. **Sans-serif** faces (Helvetica, Arial, Inter, Roboto) feel modern, clean, and neutral, and are the default for screen UI. There are also **monospace** faces (equal-width, for code) and display faces (for large headings only).

```
body { font-family: "Inter", system-ui, sans-serif; }  /* UI text */
code { font-family: "Fira Code", ui-monospace, monospace; }
```

Why it matters: the old "serif is unreadable on screen" rule is largely gone with high-DPI displays, so choose by tone, not dogma. Sans-serif remains the safe default for interface labels and small text.

Font pairing

Pairing means choosing two (rarely three) typefaces that contrast enough to create hierarchy but still feel related. The reliable recipes:

- One family, two weights (e.g. bold headings, regular body). Always

works.

- A serif heading with a sans-serif body, or the reverse.
- A distinct display face for headings with a plain, neutral body face.

```
h1, h2, h3 { font-family: "Playfair Display", serif; }  /* character */
body       { font-family: "Source Sans 3", sans-serif; } /* neutral */
```

Gotcha: avoid two faces that are subtly different (two humanist sans-serifs) — the clash reads as a mistake. Contrast should be obvious or absent. Limit a page to two families for performance and cohesion.

Typographic hierarchy

Hierarchy is the visual ranking that tells the eye what to read first. You build it with a **type scale**: a consistent ratio between sizes so h1 > h2 > body > caption feel deliberately stepped. Vary size, weight, colour, and spacing — not just size — to separate levels.

```
:root {
  --step-0: 1rem;      /* body: 16px */
  --step-1: 1.25rem;   /* h3 */
  --step-2: 1.563rem;  /* h2  (ratio ~1.25, "major third") */
  --step-3: 1.953rem;  /* h1 */
}
h1 { font-size: var(--step-3); font-weight: 700; }
p  { font-size: var(--step-0); font-weight: 400; color: hsl(0,0%,25%); }
```

Why it matters: a shared ratio (1.2, 1.25, 1.333) makes every size relate to the others, so the page looks composed rather than arbitrary.

Readable line length and line height

Two settings do most of the work for reading comfort. **Line length** (measure) should be about 45-75 characters per line; too long and the eye loses its place returning to the next line, too short and it jumps too often. The `ch` unit (width of "0") makes this easy. **Line height** (leading) for body text should be roughly 1.4-1.6 (unitless), looser than the default 1.2.

```
.article {
  max-width: 65ch;      /* comfortable measure */
  line-height: 1.6;     /* airy body leading */
  font-size: 1.125rem;  /* ~18px reads better than 16 for long text */
}
```

Gotcha: headings want *tighter* line height (1.1-1.2) because their lines are short and large; only body copy needs the loose leading.

Whitespace

Whitespace (negative space) is the empty area between and around elements. It is not wasted — it groups related things, separates unrelated ones, and gives the eye somewhere to rest. Generous spacing is the single biggest signal of a "premium" design. Use a consistent spacing scale so gaps feel systematic.

```
:root { --space-1: 8px; --space-2: 16px; --space-3: 24px; --space-4: 40px; }
.card { padding: var(--space-3); }
.stack > * + * { margin-top: var(--space-2); }  /* even vertical rhythm */
```

Why it matters: the **law of proximity** — items close together are perceived as a group. Tighten space inside a component and widen it between components to show structure without borders.

Alignment

Every element should line up with something else; a shared edge creates an invisible line that makes a layout feel ordered. Prefer strong left (or start) alignment for text-heavy interfaces, and avoid mixing many alignments on one screen. Centre small, symmetric blocks (a hero, a short form) but left-align long text and lists.

```
.form { display: grid; gap: 16px; }         /* labels + fields share
                                               a left edge automatically */
.hero { text-align: center; max-width: 50ch; margin-inline: auto; }
```

Gotcha: centred paragraphs of more than 2-3 lines are hard to read because the left edge (where the eye returns) moves on every line.

UI hierarchy and consistency

Visual hierarchy in UI means the most important action is the most prominent — through size, colour, contrast, and position. Give a screen exactly one primary button; everything else is secondary or a plain link. **Consistency** means the same thing looks and behaves the same everywhere: one button style set, one spacing scale, one set of colours. This is what a design system or component library enforces.

```
.btn           { padding: 10px 18px; border-radius: 8px; font-weight: 600; }
.btn-primary   { background: var(--primary); color: #fff; }
.btn-secondary { background: transparent; border: 1px solid var(--primary); }
```

Why it matters: consistency lets users transfer what they learned on one screen to the next, which lowers the effort of using the product.

Fitts's law

Fitts's law states the time to reach a target grows as the target gets smaller and farther away. In practice: make important, frequent controls **bigger** and put them where the pointer already is or near a screen edge (edges and corners are effectively infinite in size because the cursor stops there). Give touch targets at least ~44px so fingers can hit them.

```
.cta { min-height: 48px; padding: 14px 28px; }   /* easy to hit */
.icon-btn { width: 44px; height: 44px; }         /* min touch target */
```

Gotcha: tiny close buttons, thin links crowded together, and important actions buried in a corner of a large area all fight Fitts's law and raise error rates.

UX: simplicity

Simplicity means removing everything that is not essential so the core task is obvious. Each element on a screen competes for attention, so every extra option, field, or decoration has a cost. Prefer sensible defaults over choices, progressive disclosure (hide advanced options until needed) over showing everything, and clear copy over clever copy.

```
Signup form, minimal: email + password + one button.
Not: email, confirm email, password, confirm password, referral code,
newsletter checkbox, and a captcha, all above the fold.
```

Why it matters: fewer elements mean fewer decisions (lower cognitive load) and a faster path to the thing the user actually came to do.

UX: feedback

Feedback is the system telling the user that an action registered and what its result was. Every interaction should have a visible response: hover and focus states on controls, a loading indicator for anything that takes time, and a clear success or error message afterwards. Silence makes users click again or assume the app is broken.

```
.btn:hover  { filter: brightness(1.08); }
.btn:active { transform: translateY(1px); }
.btn:focus-visible { outline: 3px solid hsl(210,90%,70%); }
.btn[aria-busy="true"] { opacity: .6; pointer-events: none; }
```

Gotcha: keep the keyboard focus outline visible. Removing `outline` for looks makes the app unusable for keyboard and screen-reader users; style it with `:focus-visible` instead of deleting it.

UX: affordances and signifiers

An affordance is what an element lets you do; a signifier is the visual cue that advertises it. Good design makes things look like what they do: buttons look pressable (raised, filled, cursor pointer), links look clickable (colour + underline), text fields look editable (a bordered box). Breaking these conventions to be "clever" makes users hunt for what is interactive.

```
a { color: var(--primary); text-decoration: underline; }
button { cursor: pointer; }         /* signal it is clickable */
input { border: 1px solid #ccc; }   /* signal it is a field */
```

Why it matters: users rely on learned conventions. Flat "buttons" with no fill or border, or clickable text that looks like body copy, remove the signifier and hurt discoverability.

UX: user flows

A user flow is the sequence of screens and steps a person moves through to complete a goal (e.g. browse -> add to cart -> checkout -> confirm). Designing the flow, not just single screens, catches dead ends and missing states. Map the **happy path** first, then the empty, loading, and error states each step can enter.

```
Checkout flow:
  Cart -> Shipping -> Payment -> Review -> Confirmation
  branches: empty cart, payment declined, address invalid
```

Gotcha: the states that are easy to forget — empty, loading, error, and success — are exactly where users get stuck. Design them explicitly, not as an afterthought.

Grid systems

A grid is a set of evenly spaced columns (with gutters between them) that you align content to, giving a page underlying structure and rhythm. Most web grids use 12 columns because 12 divides cleanly into halves, thirds, quarters, and sixths. You already use this: Bootstrap's grid and CSS Grid both implement the same idea.

```
.layout {
  display: grid;
  grid-template-columns: repeat(12, 1fr);  /* 12-col grid */
  gap: 24px;                                /* gutter */
}
.main    { grid-column: 1 / 9; }   /* span 8 columns */
.sidebar { grid-column: 9 / 13; }  /* span 4 columns */
```

Why it matters: aligning blocks to a shared column grid is what makes independently designed sections look like one coherent page.

The 960 grid

The 960 Grid System is a classic fixed-width layout convention from the desktop era: a 960px content width divided into 12 columns (60px each) with 20px gutters (or 16 columns of 40px). 960 was chosen because it divides evenly many ways and fit the common 1024px monitors of the time.

```
960px total = 12 columns x (60px col + 20px gutter)
Each column block = 80px; last gutter trimmed.
```

Gotcha: 960 is history now — modern layouts are fluid and responsive (percentages, `fr`, `max-width`) rather than a fixed 960px. But the 12-column mental model it popularised is exactly what today's grids still use, so it is worth knowing where the convention came from.

Visual hierarchy in layout

Layout hierarchy guides the eye through the page in order of importance using size, position, contrast, and spacing. Readers of left-to-right languages tend to scan in an **F-pattern** (across the top, then down the left) for text pages or a **Z-pattern** for sparse landing pages, so put the most important content top-left and the primary action along that path. Bigger, bolder, higher-contrast, and more isolated elements read as more important.

```
.hero h1  { font-size: 3rem; }                /* size = importance */
.hero .cta { background: var(--accent); }     /* contrast draws eye */
.section  { margin-block: var(--space-4); }   /* isolation = emphasis */
```

Why it matters: you control reading order with visual weight, not just DOM order. If everything is bold and large, nothing stands out and the hierarchy collapses.

Cheatsheet

Concept	Rule of thumb	Note
Colour model	Use `hsl(h, s%, l%)`	Build palettes by tweaking l/s
Palette split	60-30-10	Dominant / secondary / accent
Complementary	Opposite hues	High contrast for CTAs
Body contrast	4.5:1 (WCAG AA)	3:1 for large text / UI
Contrast tool	WebAIM checker	Also DevTools colour picker
Body font	Sans-serif default	Serif fine on high-DPI
Pairing	2 families max	Obvious contrast, not subtle
Type scale	Ratio 1.2-1.333	Stepped, consistent sizes
Line length	45-75 chars	`max-width: 65ch`
Body leading	line-height 1.4-1.6	Headings tighter, ~1.1
Whitespace	Consistent scale	Proximity = grouping

Concept	Rule of thumb	Note
Primary action	One per screen	Size + colour + position
Fitts's law	Big, near, edges	Touch target >= 44px
Focus state	Keep `:focus-visible`	Never delete outline
Grid	12 columns + gutter	Divides many ways
960 grid	960px / 12 / 20px gap	Historic, fixed-width
Scan pattern	F (text) / Z (sparse)	Key content top-left

Common mistakes

- Picking colours as random hex codes instead of an `hsl` system, so the palette never feels cohesive.
- Light-grey text on white that fails the 4.5:1 contrast minimum and is hard to read.
- Signalling errors or states by colour alone, invisible to colour-blind users.
- Pairing two subtly different fonts so the clash looks accidental, or loading four families at once.
- Full-width paragraphs with no `max-width`, giving 120-character lines that are tiring to read.
- Default tight line height (1.2) on body text, making paragraphs feel cramped.
- Cramming a screen so full that nothing has whitespace and no hierarchy emerges.
- More than one primary button per screen, so the main action is unclear.
- Tiny click targets and controls buried far from the pointer, ignoring Fitts's law.
- Removing the focus outline for aesthetics, breaking keyboard access.
- Designing only the happy path and forgetting empty, loading, and error states.
- Making interactive things look non-interactive (flat "buttons", links styled like plain text), removing the signifier.

Quick self-check

1. In `hsl()`, which two values do you usually change to build tints and shades of one hue?
2. Name the four common colour schemes and which one gives the strongest contrast.
3. What is the WCAG AA contrast minimum for normal body text, and why is colour-only signalling a problem?

4. When would you reach for a serif face over a sans-serif?
5. What is a safe, always-works font pairing that uses only one family?
6. What is the recommended line length (measure) and body line height for comfortable reading?
7. What does the law of proximity say, and how do you use spacing to show grouping?
8. State Fitts's law in one sentence and give the minimum touch-target size.
9. Why should you style `:focus-visible` instead of removing the outline?
10. Why are web grids usually 12 columns, and where did the 960 grid come from?

Answers

1. Saturation and lightness; hold the hue fixed and vary those to get a coherent set of tints and shades.
2. Monochromatic, analogous, complementary, and triadic; complementary (opposite hues) gives the strongest contrast.
3. 4.5:1. Colour alone excludes colour-blind users and anyone in poor viewing conditions, so pair it with an icon or text.
4. When the tone is editorial, formal, or traditional, or for long-form reading; on high-DPI screens serifs read fine.
5. One family in two weights, e.g. bold for headings and regular for body text.
6. About 45-75 characters per line (roughly `65ch`), with body line height around 1.4-1.6.
7. Items placed close together are perceived as a group; tighten space inside a component and widen it between components.
8. Time to hit a target grows as it gets smaller and farther away; make key controls large (touch targets at least ~44px) and near.
9. `:focus-visible` keeps a visible focus ring for keyboard users while avoiding it on mouse clicks; deleting the outline breaks keyboard and screen-reader navigation.
10. 12 divides cleanly into halves, thirds, quarters, and sixths; the 960 grid was a fixed 960px, 12-column desktop-era convention that popularised that column model.

Chapter 8: Command Line, Git, GitHub and Deployment

What this covers

This chapter maps to the tooling sections of the bootcamp: driving your machine from a Unix-style shell, tracking your work with Git, collaborating through GitHub, and getting projects onto the public web. It covers bash navigation and file commands, the Git commit workflow and branching, the GitHub remote model (clone, push, pull, fork, pull requests), and deployment of both static sites (GitHub Pages) and dynamic Node/Express apps (a host such as Render or Railway), plus environment variables in production and the idea of CI. Version assumptions (as of 2026): Git 2.40+ (so `git switch`/`git restore` and `git init -b` exist), a POSIX shell (bash or zsh; on Windows use Git Bash or WSL), and Node.js 20+ LTS.

Key concepts

The shell, the prompt, and paths

A shell is a text program that reads commands and runs them. The prompt usually ends in `$` and shows your current directory. Paths are either absolute (start at the root `/`, e.g. `/home/riz/site`) or relative to where you are now. Three shorthands appear everywhere: `.` is the current directory, `..` is the parent, and `~` is your home directory.

```
pwd            # print working directory (where am I?)
cd ~/projects  # go to projects inside home
cd ..          # go up one level
cd -           # jump back to the previous directory
```

Why it matters: almost every command acts on "where you are now". If a file is not found, check `pwd` first before assuming the command is broken.

Listing and inspecting directories

`ls` lists what is in a directory. Flags stack: `-l` for a long listing with permissions and sizes, `-a` to include dotfiles (hidden files starting with `.`), and `-h` for human-readable sizes. Add a path to list somewhere else without moving there.

```
ls              # names only
ls -la          # long listing, including hidden files like .git and .env
ls -lh ~/logs   # long, human sizes, in another folder
```

Gotcha: files starting with a dot (`.gitignore`, `.env`) are hidden from a plain `ls`. You must pass `-a` to see them, which is exactly where config lives.

Creating files and folders

`mkdir` makes a directory; add `-p` to create nested parents in one go. `touch` creates an empty file (or updates a file's timestamp if it already exists), handy for stubbing out files before you edit them.

```
mkdir my-app
mkdir -p my-app/src/components   # makes every missing level
touch my-app/index.html my-app/app.js
```

Why it matters: without `-p`, `mkdir a/b/c` fails if `a` or `b` does not yet exist. `-p` also stays silent if the folder is already there, which is safe in scripts.

Copying, moving, and renaming

`cp` copies, `mv` moves. There is no separate "rename" command: moving a file to a new name in the same folder is a rename. Copying a folder needs `-r` (recursive). Both overwrite the destination silently unless you add `-i` to be prompted.

```
cp index.html backup.html        # copy a file
cp -r src src-old                # copy a whole folder
mv draft.md notes.md             # rename (same folder)
mv notes.md docs/                # move into docs/
```

Gotcha: `mv a b` where `b` already exists overwrites `b` with no warning. Use `mv -i` (interactive) when the target might exist and you care about it.

Deleting files (carefully)

`rm` deletes a file; `rm -r` deletes a folder and everything in it. There is no recycle bin: deletion is immediate and permanent. `rmdir` removes only an empty directory, which is a safer way to clear out folders you expect to be empty.

```
rm old.log               # delete one file
rm -r build/             # delete a folder and its contents
rmdir empty-folder       # only works if it is truly empty
```

Why it matters: `rm -rf` forces deletion without prompts and is the classic foot-gun. Double-check the path, and never run it against `/` or `~` in anger.

Viewing file contents

`cat` dumps a whole file to the screen, good for short files or feeding into a pipe. `less` opens a scrollable pager for long files (arrow keys or space to page; `q` to quit; `/word` to search). `head` and `tail` show the first or last lines, and `tail -f` follows a file live, which is how you watch logs.

```
cat package.json         # print the whole file
less server.log          # scroll a long file, press q to exit
head -n 20 data.csv      # first 20 lines
tail -f server.log       # follow new log lines as they arrive
```

Gotcha: `cat` on a huge file floods your terminal. Reach for `less` when you are not sure how big the file is.

Pipes and redirection

Every command has a standard output (stdout) and can take standard input (stdin). A pipe `|` feeds one command's output into the next. Redirection sends output to a file instead of the screen: `>` overwrites, `>>` appends, and `<` reads input from a file. These compose small tools into bigger ones.

```
ls -la | grep ".js"          # list, keep only lines mentioning .js
cat access.log | wc -l       # count the lines in the file
echo "PORT=3000" > .env      # write (overwrites the file)
echo "KEY=abc" >> .env       # append a second line
```

Why it matters: `>` clobbers the file silently, so a typo can wipe a file's contents. Use `>>` when you mean to add to what is already there.

What Git is and initialising a repo

Git is a distributed version control system: it records snapshots of your project over time so you can review history, undo changes, and collaborate. `git init` turns a folder into a repository by creating a hidden `.git` directory. Set your name and email once so commits are attributed to you.

```
git config --global user.name "Rizwan Khalid"
git config --global user.email "you@example.com"

cd my-app
git init -b main    # initialise with the default branch named main
```

Gotcha: `.git` holds the entire history. Deleting it (or copying a folder without it) discards all version history and leaves a plain directory.

The three areas and the commit workflow

Git has three states: the working directory (your files), the staging area (the index, what will go into the next commit), and the repository (committed history). `git add` stages changes; `git commit` records the staged snapshot with a message. `git status` shows what is staged, modified, or untracked.

```
git status               # what changed and what is staged
git add index.html       # stage one file
git add .                # stage everything in the current folder
git commit -m "Add landing page markup"
```

Why it matters: staging lets you craft a commit from only some of your changes. A commit should be one logical unit of work with a clear message, not a dump of everything you touched all day.

Viewing and reading history

`git log` lists commits newest first, each with a hash, author, date, and message. It is verbose by default; `--oneline` compresses each commit to one line, and `--graph` draws the branch structure. Every commit has a unique hash you can use to refer back to it.

```
git log                     # full history
git log --oneline --graph   # compact, with branch lines
git show <hash>             # see the full diff of one commit
git diff                    # unstaged changes vs the last commit
```

Gotcha: `git diff` shows only unstaged changes. Once you `git add`, use `git diff --staged` to review what is actually about to be committed.

.gitignore

Some files should never be committed: secrets (`.env`), dependencies (`node_modules`), build output, and OS/editor cruft. List them in a `.gitignore` file and Git will not track them. Patterns match names and paths; a trailing slash means a directory.

```
node_modules/
.env
dist/
*.log
.DS_Store
```

Why it matters: `.gitignore` only stops Git from tracking *new* files. If a file is already committed, you must `git rm --cached <file>` to stop tracking it, then commit that removal. This is the usual fix for an accidentally committed `.env`.

Branches

A branch is a movable pointer to a line of development. You work on a feature branch so `main` stays stable, then merge when the work is done. `git branch` lists or creates branches; modern Git uses `git switch` to move between them (`checkout` still works and does more besides).

```
git branch                      # list branches (* marks the current one)
git switch -c feature/login     # create and switch to a new branch
git switch main                 # go back to main
git branch -d feature/login     # delete a merged branch
```

Gotcha: `git switch -c` both creates and switches. Plain `git branch name` creates a branch but leaves you where you are, which surprises people.

Merging

Merging brings the commits from one branch into another. Switch to the branch you want to merge *into* (usually `main`), then `git merge` the feature branch. If the two branches changed the same lines, Git reports a conflict you must resolve by hand, then commit.

```
git switch main
git merge feature/login         # bring the feature into main
# on conflict: edit the marked files, then:
git add <resolved-file>
git commit                      # completes the merge
```

Why it matters: conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`) mark the competing versions. Delete the markers, keep the right content, and only then add and commit. Never commit the markers themselves.

checkout vs switch vs restore

Older tutorials use `git checkout` for everything: changing branches, creating them, and discarding file changes. Git 2.23 split this into two clearer commands: `git switch` for branches and `git restore` for files. Knowing both helps when reading old bootcamp material.

```
git switch main                 # (old: git checkout main)
```

```
git switch -c new-branch         # (old: git checkout -b new-branch)
git restore app.js               # discard unstaged edits to a file
git restore --staged app.js      # unstage but keep the edits
```

Gotcha: `git restore <file>` throws away your uncommitted edits with no undo. Make sure you really want to discard them before running it.

Remotes: connecting to GitHub

GitHub hosts a copy of your repo on a server so you can back it up and share it. A "remote" is a named URL for that copy; the default name is `origin`. You link a local repo to a GitHub repo you created (empty) with `git remote add`, then push.

```
git remote add origin https://github.com/you/my-app.git
git remote -v                    # show configured remotes
git push -u origin main          # first push, sets upstream tracking
```

Why it matters: the `-u` (upstream) flag on the first push links your local `main` to `origin/main`, so afterwards a bare `git push` or `git pull` knows where to go.

Cloning, pushing, and pulling

`git clone <url>` downloads an existing remote repo (with full history) and wires up `origin` for you. `git push` uploads your new commits; `git pull` downloads and merges commits others have pushed. Pull before you start work and before you push to avoid diverging.

```
git clone https://github.com/you/my-app.git
cd my-app
# ...make commits...
git pull                         # get others' changes first
git push                         # send yours up
```

Gotcha: if `git push` is rejected because the remote has commits you lack, run `git pull` (which merges), resolve any conflict, then push again. Never force push (`--force`) to a shared branch to "fix" this.

HTTPS vs SSH and authentication

GitHub no longer accepts your account password on the command line. Over HTTPS you authenticate with a Personal Access Token (PAT) in place of a password, often cached by a credential helper. Over SSH you generate a key pair and add the public key to GitHub, then use the `git@github.com:...` URL form.

```
# SSH: generate a key, then add ~/.ssh/id_ed25519.pub to GitHub settings
ssh-keygen -t ed25519 -C "you@example.com"
git remote set-url origin git@github.com:you/my-app.git
```

Why it matters: a "Support for password authentication was removed" error means you are using a password where a token or SSH key is required. Generate a PAT or set up SSH once and it stops.

Fork and pull request

To contribute to a repo you do not own, you *fork* it: GitHub makes your own copy under your account. You clone your fork, branch, commit, and push, then open a *pull request* (PR) asking the original project to merge your branch. Maintainers review, comment, and merge. This is the standard open-source flow.

```
# after forking on github.com and cloning YOUR fork:
git switch -c fix/typo
# ...edit, commit...
git push -u origin fix/typo
# then open a Pull Request on github.com from fix/typo -> upstream main
```

Gotcha: keep your fork's `main` in sync by adding the original repo as a second remote (conventionally `upstream`) and pulling from it, or your PRs will be based on stale history.

Deploying a static site with GitHub Pages

GitHub Pages serves static files (HTML, CSS, JS, images) straight from a repo for free, ideal for portfolios and simple front-ends. Push your site to GitHub, then in the repo's Settings enable Pages and choose a branch (commonly `main`) and folder (root or `/docs`). Your site appears at `https://<user>.github.io/<repo>/`.

```
git add index.html style.css
```

```
git commit -m "Portfolio site"
git push
# then: repo Settings -> Pages -> Source: main / root
```

Why it matters: Pages only serves static assets; there is no server, so it cannot run Node, Express, or a database. Because the site lives in a subpath, prefer relative links (`./style.css`, not `/style.css`) so they resolve.

Deploying a Node/Express app to a host

Dynamic apps need a real server process, so they go to a platform like Render or Railway (Heroku-style hosts). You connect your GitHub repo; the host installs dependencies with `npm install`, runs your start script, and gives you a public URL. Two things your code must do: define a `start` script and read the port from the environment.

```
{
  "scripts": { "start": "node server.js" },
  "engines": { "node": "20.x" }
}
```

```
// server.js — the host assigns a port via an env variable
const port = process.env.PORT || 3000;
app.listen(port, () => console.log(`Listening on ${port}`));
```

Gotcha: hard-coding `app.listen(3000)` breaks in production because the host picks the port for you. Always read `process.env.PORT` with a local fallback.

Environment variables in production

Secrets (API keys, database URLs, session secrets) must not live in your repo. In development you keep them in an uncommitted `.env` file loaded by the `dotenv` package. In production you never upload `.env`; instead you set the same variables in the host's dashboard (Render/Railway "Environment" settings), and your code reads them through `process.env` exactly as before.

```
require("dotenv").config();      // no-op if there is no .env (fine in prod)

const dbUrl = process.env.DATABASE_URL;
const secret = process.env.SESSION_SECRET;
```

Why it matters: the same `process.env.X` code works in both places; only the *source* of the values changes (local file vs host config). Committing `.env` leaks live credentials, so keep it in `.gitignore` and rotate any key that slips out.

Continuous Integration (CI)

CI means automatically running checks (tests, linting, a build) every time you push or open a PR, so problems are caught before merging. On GitHub this is GitHub Actions: a YAML file in `.github/workflows/` describes jobs that run on a fresh machine. A green check on a PR means the pipeline passed; a red X blocks a careless merge.

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm install
      - run: npm test
```

Gotcha: CI runs on a clean checkout, so anything that only works because of an untracked local file (like a committed-nowhere `.env` or a stray global package) will fail in CI even though it "works on my machine". That is the point.

Cheatsheet

Task	Command
Where am I	`pwd`
Change directory	`cd path` (`..` up, `~` home, `-` back)

Task	Command
List all + long	`ls -la`
Make folder(s)	`mkdir -p a/b/c`
Make empty file	`touch file.txt`
Copy file / folder	`cp a b` / `cp -r a b`
Move or rename	`mv old new`
Delete file / folder	`rm f` / `rm -r dir`
Print a file	`cat file`
Page a long file	`less file` (q to quit)
First / last lines	`head -n 20 f` / `tail -n 20 f`
Follow a log	`tail -f file`
Pipe output	`cmd1 \
Redirect / append	`cmd > file` / `cmd >> file`
Configure identity	`git config --global user.name "..."`
Start a repo	`git init -b main`
Status	`git status`
Stage	`git add .`
Commit	`git commit -m "msg"`
History (compact)	`git log --oneline --graph`
Diff (staged)	`git diff --staged`
New branch + switch	`git switch -c name`
Change branch	`git switch main`
Merge into current	`git merge feature`
Discard file edits	`git restore file`
Add remote	`git remote add origin URL`
First push	`git push -u origin main`
Clone	`git clone URL`
Get / send commits	`git pull` / `git push`
Stop tracking a file	`git rm --cached file`

Common mistakes

- Running a command in the wrong directory; always check `pwd` when a file is "missing".
- Forgetting `-a` on `ls`, so hidden files like `.env` and `.gitignore` seem gone.
- `rm -rf` on the wrong path; there is no undo and no recycle bin.
- Using `>` when you meant `>>`, overwriting a file instead of appending.
- Committing `node_modules` or `.env` because no `.gitignore` was added first.
- Thinking `.gitignore` untracks already-committed files; use `git rm --cached`.
- Writing vague commit messages ("fix", "stuff") that say nothing months later.
- Force-pushing a shared branch to resolve a rejected push instead of pulling.
- Deploying static-only GitHub Pages and expecting Express or a database to run.
- Hard-coding `app.listen(3000)` so the app fails on a host that sets `PORT`.

- Uploading `.env` to production instead of setting variables in the host panel.
- Assuming code works in CI because it works locally, ignoring untracked deps.

Quick self-check

1. What is the difference between an absolute path and a relative path, and what do `.`, `..`, and `~` mean?
2. Which `ls` flag reveals hidden dotfiles, and why does it matter for web projects?
3. What does `mkdir -p a/b/c` do that plain `mkdir` cannot?
4. What is the difference between `>` and `>>`?
5. Name Git's three areas and the command that moves a change from the first to the second.
6. A file you already committed now needs to be ignored. What two steps fix it?
7. You are on `feature`; what sequence merges it into `main`?
8. Why does `git push` sometimes get rejected, and what is the correct fix?
9. What must an Express app do to its port to run on Render or Railway?
10. Where do secrets live in development versus production, and what stays the same in the code?

Answers

1. An absolute path starts at the root `/`; a relative path is resolved from the current directory. `.` is the current directory, `..` its parent, and `~` your home directory.
2. `-a` (as in `ls -la`) shows dotfiles; config like `.env`, `.gitignore`, and `.git` is hidden without it.
3. `-p` creates every missing parent directory in the path and does not error if the directory already exists.
4. `>` redirects output to a file, overwriting its contents; `>>` appends to the end of the file instead.
5. Working directory, staging area (index), and repository; `git add` moves a change from the working directory into staging.
6. Add the file to `.gitignore`, then run `git rm --cached <file>` and commit the removal so Git stops tracking it.
7. `git switch main` then `git merge feature`; resolve any conflicts, then `git add` and `git commit`.
8. The remote has commits you do not have locally; run `git pull` to merge them (resolving conflicts), then `git push` again. Do not force push.
9. Read the port from `process.env.PORT` (with a local fallback) instead of hard-coding it, because the host assigns the port.
10. In development they live in an uncommitted `.env` file; in production they are

set in the host's environment settings. The code reads `process.env.X` the same way in both.

Chapter 9: Projects and Challenges Recap

What this covers

This chapter recaps the signature projects built across the bootcamp so you can revise by rebuilding rather than re-watching. Each project is a `### heading` below with three parts: what it teaches, the key concepts and APIs it exercises, and a short original build outline in your own words (not the instructor's code). The set is deliberately representative: two DOM/event warm-ups (Dice Game, Drum Kit), a state-machine game (Simon), a static personal site (HTML/CSS), a server-rendered app (Blog with Node/Express/EJS), a full CRUD app on a real database (To-Do List with Express + PostgreSQL), an authentication app (Secrets with Passport, bcrypt, sessions, OAuth), a React app (Keeper notes), and a Web3 DApp (DBANK / token on the Internet Computer in Motoko). Rebuild each from the outline before checking your old solution.

Key concepts

Dice Game

What it teaches. Your first taste of making a page *do* something: read the DOM, react to a page load, generate randomness, and swap image sources based on data. It cements the idea that HTML is a tree you manipulate with JavaScript.

Key concepts / APIs. `document.querySelector`, setting `.src` and other attributes, `Math.random()` and `Math.floor()`, comparing two values, updating text with `.textContent` or `innerHTML`, running code on load.

Build outline.

1. Lay out two dice images and a heading in HTML; give each die a class you can

target (e.g. `.img1`, `.img2`).

2. Prepare six dice image files named predictably, e.g. `dice1.png`..`dice6.png`.
3. In JS, write a helper that returns a random integer 1-6:

`Math.floor(Math.random() * 6) + 1`.

4. Roll both dice, then set each image's `src` to the matching file.
5. Compare the two rolls and set the heading to "Player 1 wins", "Player 2 wins",

or "Draw".

```
function roll() {
  return Math.floor(Math.random() * 6) + 1;
}
const a = roll(), b = roll();
document.querySelector(".img1").src = `images/dice${a}.png`;
document.querySelector(".img2").src = `images/dice${b}.png`;
```

Gotcha: `Math.random()` returns `[0, 1)`, so multiply by 6 and floor for 0-5, then add 1. Off-by-one here is the classic bug.

Drum Kit

What it teaches. Event listeners for both mouse and keyboard, mapping many similar elements to one handler, playing audio, and adding short-lived visual feedback. It is the leap from "one button" to "handle a set of things uniformly".

Key concepts / APIs. `addEventListener`, looping over a `querySelectorAll` NodeList, the `Audio` constructor and `.play()`, `KeyboardEvent.key`, `switch` statements, toggling a CSS class and removing it with `setTimeout`.

Build outline.

1. Render a row of buttons, one per drum sound, each labelled with a letter and

carrying that letter as its class or text.

2. Write one function that takes a key/letter and plays the matching sound via

`new Audio("sounds/x.mp3").play()` (a `switch` maps letter to file).

3. Loop over all drum buttons and attach a click listener that calls the function

with that button's letter.

4. Add a single `keydown` listener on the document that reads `event.key` and

calls the same function.

5. On each trigger, add a `pressed` class for animation and remove it after

~100ms with `setTimeout`.

```
document.querySelectorAll(".drum").forEach((btn) => {
  btn.addEventListener("click", () => play(btn.textContent));
});
document.addEventListener("keydown", (e) => play(e.key));
```

Why it matters: reusing one handler for click and keydown avoids duplicated logic. Creating a fresh `Audio` per hit lets the same sound overlap itself.

Simon (memory game)

What it teaches. Managing application **state** across time: a growing sequence, the player's progress through it, and win/lose transitions. This is where you stop reacting to single events and start running a small state machine. The course builds it with jQuery, but plain JS works identically.

Key concepts / APIs. Arrays as a sequence, tracking an index into user progress, random choice, event handlers that read game state, timed UI flashes, sound feedback, and a reset path. In jQuery: `$(...)`, `.addClass`/`.removeClass`, `.on("click")`, `setTimeout`.

Build outline.

1. Keep three pieces of state: the computer's `gamePattern` (array of colors),

the player's `userPattern` (array), and a `level`/started flag.

2. On the first key press or start, begin the game and run `nextSequence()`.

3. `nextSequence()` picks a random color, pushes it to `gamePattern`, flashes and

sounds that button, increments the level, and clears `userPattern`.

4. On each color click: push to `userPattern`, flash/sound it, then check the

answer at the just-clicked index.

5. If it matches and the player has reproduced the whole pattern, wait briefly

and call `nextSequence()` for the next round.

6. If it mismatches, play a fail sound, flash a "game over" style, and reset all

state so a new press starts over.

```
function checkAnswer(i) {
  if (userPattern[i] !== gamePattern[i]) return gameOver();
  if (userPattern.length === gamePattern.length) {
    setTimeout(nextSequence, 800); // player finished this round
  }
}
```

Gotcha: compare only the index the player just clicked, not the whole array, so you catch a wrong press immediately instead of after the full sequence.

Personal Site (HTML/CSS)

What it teaches. Structuring a real page with semantic HTML and styling it with CSS: the box model, layout with Flexbox, typography, responsive behaviour, and deploying a static site. No JavaScript required — it is about clean

markup and visual polish.

Key concepts / APIs. Semantic elements (`header`, `nav`, `main`, `section`, `footer`), the CSS box model (margin/border/padding/content), Flexbox (`display: flex`, `justify-content`, `align-items`), media queries for responsiveness, web fonts, and static hosting (e.g. GitHub Pages).

Build outline.

1. Sketch the sections: hero/intro, about, skills or projects, contact.
2. Write semantic HTML top to bottom before touching CSS, so the document reads

well unstyled.

3. Add a stylesheet: set a base font and colours on `body`, then style section by

section. Use a container with a max-width centred via `margin: 0 auto`.

4. Use Flexbox for the nav bar and any card rows; let items wrap on small screens.
5. Add one or two media queries (`@media (max-width: 600px)`) to stack columns and

shrink the hero on phones.

6. Deploy as static files to a host like GitHub Pages.

```
.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
@media (max-width: 600px) {
  .cards { flex-direction: column; }
}
```

Why it matters: getting semantic structure right first makes the CSS simpler and the page accessible by default. Layout is Flexbox for rows; reach for Grid only when you need two-dimensional layout.

Blog Web App (Node / Express / EJS)

What it teaches. Moving from static pages to a server that renders HTML per request. You learn routing, handling form posts, passing data into templates, and keeping data in server memory before a database is introduced.

Key concepts / APIs. Express (`app.get`, `app.post`), the EJS view engine (`res.render`, `<%= %>` output vs `<% %>` logic, partials with `<%- include %>`), `express.urlencoded` for form bodies, `express.static` for CSS/assets, and route parameters (`/posts/:id`).

Build outline.

1. Set up Express, point the view engine at EJS, serve a `public` folder, and add

`express.urlencoded({ extended: true })` to read form posts.

2. Keep posts in an in-memory array of `{ id, title, body }` (a database comes

later).

3. `GET /` renders a home template that loops over posts and links each to its

detail page.

4. `GET /compose` shows a form; `POST /compose` reads `req.body`, builds a post,

pushes it, and redirects home.

5. `GET /posts/:id` finds the post by `req.params.id` and renders a detail view;

return a 404 view if not found.

6. Factor the header and footer into EJS partials so every page shares them.

```
app.post("/compose", (req, res) => {
  posts.push({ id: nextId++, title: req.body.title, body: req.body.body });
```

```
  res.redirect("/");
});
app.get("/posts/:id", (req, res) => {
  const post = posts.find((p) => p.id === Number(req.params.id));
  post ? res.render("post", { post }) : res.status(404).render("404");
});
```

Gotcha: use `<%= %>` for text (it escapes HTML and prevents injection) and `<%- %>` only for trusted markup like `include`. Redirect after a POST so a refresh doesn't resubmit the form.

To-Do List (Express + PostgreSQL CRUD)

What it teaches. Persisting data in a real relational database and wiring the four CRUD operations end to end: create, read, update, delete. This is the first project where data survives a server restart.

Key concepts / APIs. PostgreSQL, the `pg` client/`Pool`, parameterised queries (`$1`, `$2`), SQL `INSERT/SELECT/UPDATE/DELETE`, mapping HTTP verbs to CRUD, and rendering DB rows through EJS. As of 2026, prefer `async/await` with `pool.query` over callbacks.

Build outline.

1. Create a `pg` `Pool` from a connection string in `.env`; create an `items`

table with `id SERIAL PRIMARY KEY` and a `title TEXT`.

2. `GET /` runs `SELECT * FROM items ORDER BY id` and renders the list.
3. `POST /add` inserts `req.body.title` with a parameterised query, then redirects

home.

4. `POST /edit` updates a row by id; `POST /delete` deletes by id. (With plain

HTML forms you POST these; a JSON API would use PUT/PATCH and DELETE.)

5. Always pass user input as query parameters, never string-concatenated into SQL.

```
// Read
const { rows } = await pool.query("SELECT * FROM items ORDER BY id");
// Create (parameterised — safe from SQL injection)
await pool.query("INSERT INTO items (title) VALUES ($1)", [req.body.title]);
// Delete
await pool.query("DELETE FROM items WHERE id = $1", [req.body.id]);
```

Gotcha: never build SQL by concatenating `req.body` into the string — use `$1` placeholders so the driver escapes values and blocks SQL injection. Close or pool connections; a `Pool` reuses them for you.

Secrets (auth app: Passport, bcrypt, sessions, OAuth)

What it teaches. Real authentication built up in layers: hashed passwords, server-side sessions to stay logged in, Passport strategies to structure it, and finally "Log in with Google" via OAuth. It ties together security concepts from Chapter 5 into one working app.

Key concepts / APIs. `bcrypt.hash/bcrypt.compare`, `express-session`, `passport`, `passport-local`, `passport.serializeUser/deserializeUser`, `passport-google-oauth20`, `req.isAuthenticated()`, `req.login/req.logout`, and route guards. Secrets live in `.env`.

Build outline.

1. Start with a `users` table/collection and a register route: hash the password

with `bcrypt.hash(pw, 12)` before saving; never store plaintext.

2. Add `express-session`, then `passport.initialize()` and `passport.session()`

after it.

3. Configure the local strategy: look up the user, `bcrypt.compare` the password,

and call `done(null, user)` or `done(null, false)`.

4. Implement `serializeUser` (store just the id) and `deserializeUser` (reload the

user from the id).

5. Protect the "secrets" page with a guard: if `req.isAuthenticated()`, render;

else redirect to login.

6. Add Google OAuth with `passport-google-oauth20`: on the callback, find-or-create

a local account keyed by the Google profile id. Allow accounts with no password hash for OAuth-only users.

```
app.get("/secrets", (req, res) => {
  if (req.isAuthenticated()) return res.render("secrets");
  res.redirect("/login");
});
```

Gotcha: the OAuth code exchange and client secret must stay on the server, not in the browser. Put `express.session` before Passport, or logins won't persist.

Keeper App (React notes app)

What it teaches. Composing a React UI from components, lifting state to a parent, passing data down as props and callbacks up, rendering a list with keys, and using controlled form inputs. It is the React counterpart to the earlier vanilla-JS DOM work.

Key concepts / APIs. Function components, `useState`, props and `children`, lifting state up, `array.map` with a stable `key`, controlled inputs (`value`/`onChange`), immutable updates with the spread operator and `filter`/`map`, and passing an `onDelete(id)` callback to children.

Build outline.

1. Build three components: `App` (owns the notes array), `CreateArea` (the form),

and `Note` (one card).

2. Hold `notes` in `App` via `useState([])`. Render them with

`notes.map((n) => <Note key={n.id} ... />)`.

3. `CreateArea` keeps its own controlled state for title/content, and on submit

calls an `onAdd` prop passed from `App`.

4. `App`'s add handler appends immutably: `setNotes((prev) => [...prev, newNote])`.

5. `Note` gets an `onDelete` prop; clicking delete calls `onDelete(id)`, and `App`

removes it with `setNotes((prev) => prev.filter((n) => n.id !== id))`.

6. Give each note a stable id (a counter or `crypto.randomUUID()`), not the array

index, for its key.

```
function App() {
  const [notes, setNotes] = useState([]);
  const addNote = (note) => setNotes((prev) => [...prev, note]);
  const deleteNote = (id) =>
    setNotes((prev) => prev.filter((n) => n.id !== id));
  return (
    <>
      <CreateArea onAdd={addNote} />
      {notes.map((n) => (
        <Note key={n.id} note={n} onDelete={deleteNote} />
      ))}
    </>
  );
}
```

Why it matters: state lives in the common parent (`App`) because both the form and the list need it. Children stay "dumb" — they receive data and report events up through callbacks.

DBANK / Token DApp (Internet Computer, Motoko)

What it teaches. Building a decentralised app: writing backend logic as a **canister** smart contract in Motoko, persisting state on-chain, exposing update and query methods, and calling them from a web frontend. DBANK is a simple interest-bearing balance; the token project extends it to transferable balances and ownership. This is the Web3 capstone from Chapter 6.

Key concepts / APIs. The `dfx` CLI (`dfx start`, `dfx deploy`), Motoko `actor` canisters, `stable` variables for upgrade-safe state, `public func` (update) vs `public query func` (fast read-only), the `Principal` type for identity, a `HashMap` for balances, and the generated JS actor used from the frontend. State version assumption: dfx / DFINITY SDK as of 2026.

Build outline (DBANK).

1. Define an `actor` with a `stable var currentValue : Float`.
2. Add update methods `topUp(amount)` and `withdraw(amount)` that mutate

`currentValue` (guard withdraw against overdraft).

3. Add a `query func checkBalance()` returning `currentValue` for cheap reads.
4. Optionally compute compound interest based on elapsed time on each call.
5. Deploy locally with `dfx deploy`; call the methods from an HTML/JS frontend

through the generated actor.

Build outline (Token).

1. Keep balances in a `HashMap<Principal, Nat>` and a `stable` owner principal.
2. On first deploy, mint the supply to the owner.
3. Add `balanceOf(who)` as a query, and `transfer(to, amount)` as an update that

checks the caller's balance via `msg.caller` before moving funds.

4. Expose a faucet or claim method for the demo, then wire buttons in the frontend

to the actor's methods.

```
actor DBank {
  stable var currentValue : Float = 0;

  public func topUp(amount : Float) : async () {
    currentValue += amount;
  };

  public query func checkBalance() : async Float {
    return currentValue;
  };
};
```

Gotcha: mark persistent fields `stable`, or their values reset on every canister upgrade. Use `query` methods for reads (fast, not consensus-committed) and plain `public func` for state changes; a `HashMap` itself is not stable, so serialise it to a stable array in `preupgrade`/`postupgrade` hooks.

Cheatsheet

Project	Core API / idea
Dice Game	`Math.floor(Math.random()*6)+1`, set `.src`
Drum Kit	`addEventListener`, `new Audio().play()`
Simon	sequence array + index, state machine
Personal Site	semantic HTML, Flexbox, media query
Blog	Express routes, EJS `res.render`

Project	Core API / idea
To-Do List	`pool.query("... $1", [val])` CRUD
Secrets	bcrypt + session + Passport + OAuth
Keeper	lift state, props down, callbacks up
DBANK / Token	Motoko `actor`, `stable`, `query func`
Escape HTML in EJS	`<%= value %>` not `<%- value %>`
React immutable add	`setX(prev => [...prev, item])`
React immutable remove	`prev.filter(x => x.id !== id)`
Deploy canister	`dfx start` then `dfx deploy`

Common mistakes

- Off-by-one in the dice roll: multiplying `Math.random()` by 6 but forgetting `+ 1`, so you can never roll a 6 (or you index a missing image).
- Attaching the drum handler to only one button instead of looping the whole NodeList, or forgetting the document-level `keydown` listener.
- In Simon, comparing the entire pattern instead of just the index the player clicked, so you miss a wrong press until the round ends.
- Styling a personal site before writing semantic HTML, ending up with div soup that is hard to lay out and inaccessible.
- Forgetting `express.urlencoded` in the Blog app, so `req.body` is undefined on form posts; or not redirecting after POST, causing resubmits on refresh.
- Concatenating user input into SQL in the To-Do app instead of using `$1` placeholders (SQL injection).
- Putting the OAuth client secret or code exchange in the browser, or placing the session middleware after Passport in the Secrets app.
- Mutating React state directly in Keeper (`notes.push(...)`) instead of setting a new array; using the array index as a `key`.
- Not marking Motoko persistent state `stable`, so balances reset on every canister upgrade.

Quick self-check

1. Why does the dice roll add `+ 1` after `Math.floor(Math.random() * 6)`?
2. In the Drum Kit, how do you make one handler serve both clicks and key presses?
3. In Simon, which index should you compare on each player click, and why?
4. Why write semantic HTML before CSS on the personal site?
5. What middleware lets the Blog app read `req.body` from a form POST, and why redirect after a POST?
6. How does the To-Do app avoid SQL injection when inserting a title?
7. In the Secrets app, where must the OAuth code exchange happen, and why?

8. In Keeper, where does the notes array live and how do you add one immutably?
9. What does the `stable` keyword do in a Motoko canister?
10. What is the difference between a Motoko `query func` and a plain `public func`?

Answers

1. `Math.random()` returns `[0, 1)`, so `floor(*6)` gives 0-5; adding 1 shifts it to 1-6, matching the dice faces.
2. Write one `play(key)` function and call it from both a click listener on each button and a single `keydown` listener on the document.
3. The index of the button the player just clicked (`userPattern.length - 1`), so a wrong press is caught immediately rather than at the end of the sequence.
4. Correct semantic structure makes the CSS simpler, keeps the page accessible, and reads sensibly even before any styling is applied.
5. `express.urlencoded({ extended: true })` parses form bodies into `req.body`. Redirecting after POST follows the Post/Redirect/Get pattern so a refresh doesn't resubmit the form.
6. It uses parameterised queries with `$1` placeholders and passes values in an array, so the driver escapes them instead of concatenating into the SQL string.
7. On the server. The client secret and the one-time code must never be exposed in the browser; only the server exchanges the code for tokens.
8. In the parent `App` component via `useState`. Add immutably with `setNotes(prev => [...prev, newNote])` rather than mutating the array.
9. It marks a variable's value to be preserved across canister upgrades; without it the value resets to its initial state on redeploy.
10. A `query func` is read-only and fast (not consensus-committed), while a plain `public func` can change state and goes through consensus.